

2

Structured Query Language

Syllabus

SQL : Characteristics and Advantages, SQL Data Types and Literals, DDL, DML, DCL, TCL, SQL Operators. **Tables** : Creating, Modifying, Deleting, Updating. **SQL DML Queries** : SELECT Query and clauses, Index and Sequence in SQL. **Views** : Creating, Dropping, Updating using Indexes, Set Operations, Predicates and Joins, Set membership, Tuple Variables, Set comparison, Ordering of Tuples, Aggregate Functions, SQL Functions, Nested Queries.

Contents

- 2.1 Introduction to Structured Query Language (SQL)
- 2.2 SQL Data Types and Literals **Nov.-17,** Marks 5
- 2.3 DDL, DML, DCL and TCL Structure
- 2.4 Tables
- 2.5 SQL DML Queries
- 2.6 Logical Operators
- 2.7 String Operations
- 2.8 The BETWEEN Operator
- 2.9 Built-In Functions
- 2.10 NULL Values
- 2.11 EXISTS, NOT EXISTS and UNIQUE
- 2.12 Defining Constraints
- 2.13 Renaming Attributes
- 2.14 Tuple Variables

2.15	Schema Change Statements	Nov.-18,	Marks 5
2.16	Indexes	Aug. 17, Dec. 18,	
		May 19,	Marks 5
2.17	Aggregate Functions	Nov.-17,	Marks 2
2.18	Set Operations		
2.19	Nested Queries		
2.20	Join Operation	Nov.-17,19, May-19,	Marks 6
2.21	Views	July-18, Oct.-18,	
		Nov.-19,	Marks 8
2.22	Examples Based on SQL	Aug 17,	
		Dec. 17,18,	
		May 18,19,	
		Oct 18,19,	Marks 5

2.1 Introduction to Structured Query Language (SQL)

- SQL stands for **Structured Query Language**.
- It is the language of databases and almost all companies use databases to store their data.
- SQL makes use of query. A Query is a set of instruction given to the database management system. It tells any database what information we would like to get from the database.
- SQL is **case- insensitive**. However it has become standard in SQL community to use **all capital** letters for SQL keywords.
- SQL is a standard language for Relational Database Management System (RDBMS).
- There are various RDBMS software that are popularly used. It includes MySQL, Oracle, MS ACCESS, Microsoft SQL Server, Sybase and so on.

2.1.1 Characteristics and Advantages

- 1) SQL is a standard computer language for creating and manipulating databases.
- 2) SQL is very simple and easy to learn.
- 3) SQL allows the users to create, update, delete and retrieve data from the database.
- 4) SQL is used to create view, stored procedures and functions in a database.
- 5) SQL allows the users to set the permissions on the tables, procedures and views in the database.

2.2 SQL Data Types and Literals

SPPU : Nov.-17, Marks 5

Various data types used in SQL are -

1) Numeric data types

- Integer numbers : INT, INTEGER, SMALLINT, BIGINT
- Floating-point (real) numbers : REAL, DOUBLE, FLOAT
- Fixed-point numbers : DECIMAL(n,m), DEC(n,m), NUMERIC(n,m), NUM(n,m)

2) Character-string data types

- Fixed length : CHAR(n), CHARACTER(n)
- Varying length : VARCHAR(n), CHAR VARYING(n), CHARACTER VARYING(n), LONG VARCHAR

3) Large object data types

- Characters : CLOB, CHAR LARGE OBJECT, CHARACTER LARGE OBJECT

- Bits : BLOB, BINARY LARGE OBJECT

4) Boolean data type

- Values of TRUE or FALSE or NULL

5) DATE data type

- Ten positions
- Components are YEAR, MONTH, and DAY in the form YYYY-MM-DD

6) Additional Data type

a) TIMESTAMP data type

It includes the DATE and TIME fields.

b) INTERVAL data type

It specifies a relative value that can be used to increment or decrement an absolute value of a date, time, or timestamp.

2.3 DDL, DML, DCL and TCL Structure

There are four types of SQL commands -

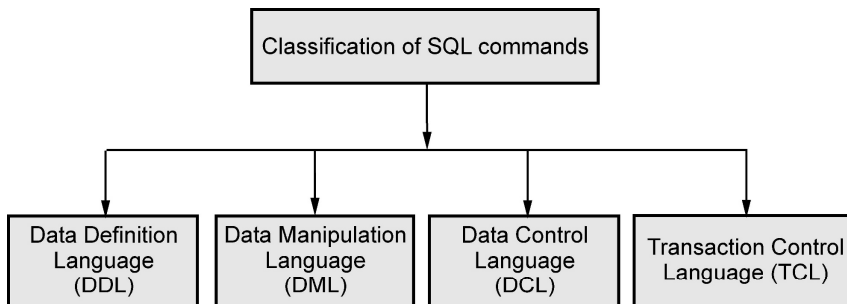


Fig. 2.3.1

Data Definition Language

Sr.No.	Command	Purpose
1.	CREATE	This command is used to create database, tables, views or any other database objects.
2.	ALTER	It modifies the existing tables.
3.	DROP	This command deletes complete table, view or any other database object.

Data Manipulation Language

Sr. No.	Command	Purpose
1.	SELECT	This command is used to retrieve either all or desired records from one or more tables.
2.	INSERT	For inserting the records in the table, this command is used.
3.	UPDATE	For updating one or more fields of the table, this command is used.
4.	DELETE	This command is used for deleting the desired record

Data Control Language

Sr. No.	Command	Purpose
1.	GRANT	This command is used to give access rights or privileges to the database.
2.	INVOKE	The revoke command removes user access rights or privileges to the database objects.

Transaction Control Language (TCL)

Sr. No.	Command	Purpose
1.	COMMIT	This command is used to save permanently any transaction to database
2.	ROLLBACK	The ROLLBACK command is used to undo transactions that have not already saved to database.

Difference between DDL and DML

DDL	DML
DDL stands for Data Definition Language.	DML stands for Data Manipulation Language.
DDL commands are used to define database structure.	DML commands are used for managing data within the database.
It works on whole table.	It works on one or more rows.
It cannot be classified further.	It can be classified as – procedural and non-procedural language.
Changes made by DDL commands cannot be rolled back.	Changes by DML commands can be rolled back.
Example - CREATE, ALTER, DROP commands.	Example - SELECT, INSERT, UPDATE, and DELETE commands.

- **Terminology :** In relational model we use the terms relation tuple and attribute.

Same terms are used in SQL as follows

Relation Model	SQL
Relation	Table
Tuple	Row
Attribute	Column

2.4 Tables

2.4.1 Creating Table

- A database can be considered as a container for tables and a table is a grid with rows and columns to hold data.
- Individual statements in SQL are called **queries**.
- We can execute SQL queries for various tasks such as creation of tables, insertion of data into the tables, deletion of record from table, and so on.

In this section we will discuss how to create a table.

Step 1 : We normally create a database using following SQL statement

Syntax

```
CREATE DATABASE database_name;
```

Example

```
CREATE DATABASE Person_DB
```

Step 2 : The table can be created inside the database as follows –

```
CREATE TABLE table_name (  
    col1_name datatype,  
    col2_name datatype,  
    ...  
    coln_name datatype  
);
```

Example

```
CREATE TABLE person_details (  
    AadharNo int,  
    FirstName VARCHAR(20),  
    MiddleName VARCHAR(20),  
    LastName VARCHAR(20),  
    Address VARCHAR(30),  
    City VARCHAR(10)  
)
```

The blank table will be created with following structure

Person_details

AadharNo	FirstName	MiddleName	LastName	Address	City
----------	-----------	------------	----------	---------	------

2.4.2 Insertion of Data into the Table

- We can insert data into the table using INSERT statement.

Syntax

```
INSERT INTO table_name (col1,col2,...,coln)
VALUES (value1,value2,...,valuen)
```

Example

```
INSERT INTO person_details (AadharNo, FirstName, MiddleName, LastName, Address, City)
VALUES (111,'AAA','BBB','CCC','M.G. Road','Pune')
```

The above query will result into –

AadharNo	FirstName	MiddleName	LastName	Address	City
111	AAA	BBB	CCC	M.G. Road	Pune

2.4.3 Modifying the Record from the Table

- For modifying the existing record of a table, update query is used.

Syntax

```
UPDATE table_name
SET col1=value1, col2=value2,...
WHERE condition;
```

The WHERE command is used to specify some condition. Based on this condition the data present in the table can be displayed or can be updated or deleted.

Example

Consider following table

Person_details table

AadharNo	FirstName	MiddleName	LastName	Address	City
111	AAA	BBB	CCC	M.G. Road	Pune
222	DDD	EEE	FFF	Shivaji nagar	Pune
333	GGG	HHH	III	Chandani Chowk	Delhi
444	JJJ	KKK	LLL	Viman Nagar	Mumbai

If we execute following query

```
UPDATE person_details
SET city='Chennai'
WHERE AadharNo=333
```

The result will be

AadharNo	FirstName	MiddleName	LastName	Address	City
111	AAA	BBB	CCC	M.G. Road	Pune
222	DDD	EEE	FFF	Shivaji nagar	Pune
333	GGG	HHH	III	Chandani chowk	Chennai
444	JJJ	KKK	LLL	Viman nagar	Mumbai

2.4.4 Deleting Record from the Table

- We can delete one or more records based on some condition. The syntax is as follows –

Syntax

```
DELETE FROM table_name WHERE condition;
```

Example

```
DELETE FROM person_details
WHERE AadharNo=333
```

The result will be -

AadharNo	FirstName	MiddleName	LastName	Address	City
111	AAA	BBB	CCC	M.G. road	Pune
222	DDD	EEE	FFF	Shivaji nagar	Pune
444	JJJ	KKK	LLL	Viman nagar	Mumbai

We can delete all the records from table. But in this deletion, all the records get deleted without deleting table. For that purpose the SQL statement will be

```
DELETE FROM person_details;
```

2.5 SQL DML Queries

- DML stands for Data manipulation Language
- The basic operations under DML queries are SELECT, INSERT, UPDATE, and DELETE

We have already discussed INSERT, DELETE and UPDATE operations in previous section. Let us now discuss the SELECT operation.

2.5.1 SELECT Query

- The Select statement is used to fetch the data from the database table.
- The result returns the data in the form of table. These result tables are called **resultsets**.
- We can use the keyword **DISTINCT**. It is an optional keyword indicating that the answer should **not contain duplicates**. Normally if we write the SQL without DISTINCT operator then it does not eliminate the duplicates.

Syntax

```
SELECT col1, col2, ...,coln FROM table_name;
```

Example

```
SELECT AadharNo, FirstName, Address, City FROM person_details
```

The result of above query will be

AadharNo	FirstName	City
111	AAA	Pune

- If we want to select all the records present in the table we make use of * character.

Syntax

```
SELECT * FROM table_name;
```

Example

```
SELECT * FROM person_details;
```

The above query will result into

AadharNo	FirstName	MiddleName	LastName	Address	City
111	AAA	BBB	CCC	M.G. road	Pune

- **Use of DISTINCT Keyword:** The keyword DISTINCT is used along with the SELECT statements.
- It is used to obtain unique values from the table. This query does not allow duplication of element.

Syntax :

```
SELECT DISTINCT Column-name FROM table-name;
```

Example:

Consider following database table **Student**

Roll No	Name	City
1	Ankita	Pune
2.	Rohit	Hyderabad
3.	Prajakta	Chennai
4.	Sunil	Pune
5.	Sharda	Chennai

SQL Statement

```
SELECT DISTINCT City
FROM Student;
```

This will result into

Pune
Hyderabad
Chennai

2.5.2 WHERE

The WHERE command is used to specify some condition. Based on this condition the data present in the table can be displayed or can be updated or deleted.

Syntax

```
SELECT col1,col2, ...,coln
FROM table_name
WHERE condition;
```

Example

Consider following table

AadharNo	FirstName	MiddleName	LastName	Address	City
111	AAA	BBB	CCC	M.G. road	Pune
222	DDD	EEE	FFF	Shivaji nagar	Pune
333	GGG	HHH	III	Chandani chowk	Delhi
444	JJJ	KKK	LLL	Viman nagar	Mumbai

If we execute the following query

```
SELECT AadharNo
FROM person_details
WHERE city='Pune';
```

The result will be –

AadharNo	City
111	Pune
222	Pune

If we want records of all those person who live in city Pune then we can write the query using WHERE clause as

```
SELECT *
FROM person_details
WHERE city='Pune';
```

The result of above query will be

AadharNo	FirstName	MiddleName	LastName	Address	City
111	AAA	BBB	CCC	M.G. road	Pune
222	DDD	EEE	FFF	Shivaji nagar	Pune

2.5.3 Clauses

Most commonly used clauses in SQL statements are Order by, Group by and Having. Let us discuss them along with syntax and examples.

(1) Order By

- Many times we need the records in the table to be in sorted order.
- If the records are arranged in increasing order of some column then it is called **ascending order**.
- If the records are arranged in decreasing order of some column then it is called **descending order**.
- For getting the sorted records in the table we use **ORDER BY** command.
- The ORDER BY keyword sorts the records in ascending order by default.

Syntax

```
SELECT col1, col2,...,coln
FROM table_name
ORDER BY col1,col2,... ASC|DESC
```

Here ASC is for ascending order display and DESC is for descending order display.

Example

Consider following table

AadharNo	FirstName	MiddleName	LastName	Address	City
111	AAA	BBB	CCC	M.G. road	Pune
222	DDD	EEE	FFF	Shivaji nagar	Pune
333	GGG	HHH	III	Chandani chowk	Delhi
444	JJJ	KKK	LLL	Viman nagar	Mumbai

```
SELECT *
FROM person_details
ORDER BY AadharNo DESC;
```

The above query will result in

AadharNo	FirstName	MiddleName	LastName	Address	City
444	JJJ	KKK	LLL	Viman nagar	Mumbai
333	GGG	HHH	III	Chandani chowk	Delhi
222	DDD	EEE	FFF	Shivaji nagar	Pune
111	AAA	BBB	CCC	M.G. road	Pune

(2) Group By

- The GROUP BY clause is a SQL command that is used to group rows that have the same values.
- The GROUP BY clause is used in the SELECT statement.
- Optionally it is used in conjunction with aggregate functions.
- The queries that contain the GROUP BY clause are called grouped queries
- This query returns a single row for every grouped item.
- **Syntax :**

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
```

- **Example :** Consider the Student table as follows -

sid	sname	marks	city
1	AAA	60	Pune
2	BBB	70	Mumbai
3	CCC	90	Pune
4	DDD	55	Mumbai

Query : Find the total marks of each student in each city

```
SELECT SUM(marks), city
FROM Student
GROUP BY city
```

The result will be as follows –

SUM(marks)	city
150	Pune
125	Mumbai

(3) Having

- HAVING filters records that work on summarized GROUP BY results.
- HAVING applies to summarized group records, whereas WHERE applies to individual records.
- Only the groups that meet the HAVING criteria will be returned.
- HAVING requires that a GROUP BY clause is present.
- WHERE and HAVING can be in the same query.
- **Syntax :**

```
SELECT column-names
FROM table-name
WHERE condition
GROUP BY column-names
HAVING condition
```

- **Example :** Consider the Student table as follows -

sid	sname	marks	city
1	AAA	60	Pune
2	BBB	70	Mumbai
3	CCC	90	Pune
4	DDD	55	Mumbai
5	EEE	84	Chennai

Query : Find the total marks of each student in the city named 'Pune' and 'Mumbai' only

```
SELECT SUM(marks), city
FROM Student
GROUP BY city
HAVING city IN('Pune','Mumbai')
```

- The result will be as follows –

SUM(marks)	city
150	Pune
125	Mumbai

2.6 Logical Operators

- Using WHERE clause we can use the operators such as AND, OR and NOT.
- AND operator displays the records if all the conditions that are separated using AND operator are true.
- OR operator displays the records if any one of the condition separated using OR operator is true.
- NOT operator displays a record if the condition is NOT TRUE.

Consider following table

AadharNo	FirstName	MiddleName	LastName	Address	City
111	AAA	BBB	CCC	M.G. road	Pune
222	DDD	EEE	FFF	Shivaji nagar	Pune

333	GGG	HHH	III	Chandani chowk	Delhi
444	JJJ	KKK	LLL	Viman nagar	Mumbai

Syntax of AND

```
SELECT col1, col2, ...  
FROM table_name  
WHERE condition1 AND condition2 AND condition3 ...;
```

Example of AND

If we execute following query –

```
SELECT AadharNo, FirstName, City  
FROM person_details  
WHERE AadharNo=222 AND City='Pune';
```

The result will be –

AadharNo	FirstName	City
222	DDD	Pune

Syntax of OR

```
SELECT col1, col2, ...  
FROM table_name  
WHERE condition1 OR condition2 OR condition3 ...;
```

Example of OR

```
SELECT AadharNo, FirstName, City  
FROM person_details  
WHERE City='Pune' OR City='Mumbai';
```

The result will be –

AadharNo	FirstName	City
111	AAA	Pune
222	DDD	Pune
444	JJJ	Mumbai

Syntax of NOT

```
SELECT col1, col2, ...  
FROM table_name  
WHERE NOT condition;
```

Example of NOT

```
SELECT AadharNo, FirstName, City  
FROM person_details  
WHERE NOT City='Pune';
```

The result will be

AadharNo	FirstName	City
333	GGG	Delhi
444	JJJ	Mumbai

2.7 String Operations

- For string comparisons, we can use the comparison operators =, <, >, <=, >=, <> with the ordering of strings determined alphabetically as usual.
- SQL also permits a variety of functions on character strings such as concatenation using operator ||, extracting substrings, finding length of string, converting strings to upper case (using function **upper(s)**) and lowercase (using function **lower(s)**), removing spaces at the end of string (using function (trim(s)) and so on.
- Pattern matching can also be performed on strings using two types of special characters –
 - Percent(%) : It matches zero, one or multiple characters
 - Underscore(_) : The _ character matches any single character.
- The percentage and underscore can be used in combinations.
- Patterns are case sensitive. That means upper case characters do not match lowercase characters or vice versa.
- For instance :**
 - 'Data%' matches any string beginning with "Data", For instance it could be with "Database", "DataMining", "DataStructure"
 - '___' matches any string of exactly three characters.
 - '___ %' matches any string of at least length 3 characters.
- The **LIKE** clause can be used in **WHERE** clause to search for specific patterns.
- For example – Consider following Employee Database**

EmpID	EmpName	Department	Date_of_Join
1	Sunil	Marketing	1-Jan
2	Mohsin	Manager	2-Jan
3	Supriya	Manager	3-Jan
4	Sonia	Accounts	4-Jan
5	Suraj	Sales	5-Jan
6	Archana	Purchase	6-Jan

(1) Find all the employee with EmpName starting with “s”**SQL Statement :**

```
SELECT * FROM Employee  
WHERE EmpName LIKE 's%'
```

Output

EmpID	EmpName	Department	Date_of_Join
1	Sunil	Marketing	1-Jan
3	Supriya	Manager	3-Jan
4	Sonia	Accounts	4-Jan
5	Suraj	Sales	5-Jan

(2) Find the names of employee whose name begin with S and end with a**SQL Statement :**

```
SELECT EmpName FROM Employee  
WHERE EmpName LIKE 'S%a'
```

Output

EmpName
Supriya
Sonia

(3) Find the names of employee whose name begin with S and followed by exactly four characters

```
SELECT EmpName FROM Employee  
WHERE EmpName LIKE 'S_ _ _ _ '
```

Output

EmpName
Sunil
Sonia
Suraj

2.8 The BETWEEN Operator

- The **between** operator can be used to simplify the where clause which is used to denote the value be less than or equal to some value and greater than or equal to some other value.
- For example – if we want the names of the students whose marks are between 80 and 90 then SQL statement will be

```
SELECT name  
FROM Students  
WHERE marks BETWEEN 80 and 90;
```

2.9 Built-In Functions

- In SQL a built-in function is a piece for programming that takes zero or more inputs and returns a value.
- An example of a built-in functions is ABS(), which when given a value calculates the absolute (non-negative) value of the number.

Query

```
SELECT ABS(-9) Result;
```

Output

Result
9

Mathematical Functions

Function	Value Returned
ABS (m)	Absolute value of m
MOD (m, n)	Remainder of m divided by n
POWER (m, n)	m raised to the nth power
ROUND (m [, n])	m rounded to the nth decimal place
TRUNC (m [, n])	m truncated to the nth decimal place
SIN (n)	sine (n)

COS (n)	cosine (n)
TAN (n)	tan (n)
SQRT (n)	positive square root of n
EXP (n)	e raised to the power n
LOG (n2, n1)	logarithm of n1, base n2
CEIL (n)	smallest integer greater than or equal to n
FLOOR (n)	greatest integer smaller than or equal to n
SIGN (n)	-1 if n < 0, 0 if n = 0, and 1 if n > 0

String Functions

Function	Value Returned
INITCAP (s)	First letter of each word is changed to uppercase and all other letters are in lower case.
LOWER (s)	All letters are changed to lowercase.
UPPER (s)	All letters are changed to uppercase.
CONCAT (s1, s2)	Concatenation of s1 and s2. Equivalent to s1 s2
LTRIM (s [, set])	Returns s with characters removed up to the first character not in set; defaults to space
RTRIM (s [, set])	Returns s with final characters removed after the last character not in set; defaults to space
REPLACE (s, search_s [, replace_s])	Returns s with every occurrence of search_s in s replaced by replace_s; default removes search_s
SUBSTR (s, m [, n])	Returns a substring from s, beginning in position m and n characters long; default returns to end of s.
LENGTH (s)	Returns the number of characters in s.

For example

```
SELECT CONCAT('Bill', 'Gates') as "NAME"
```

Date and Time Functions

Function	Value Returned
ADD_MONTHS (d, n)	Date d plus n months
LAST_DAY (d)	Date of the last day of the month containing d
MONTHS_BETWEEN (d, e)	Number of months by which e precedes d
NEW_TIME (d, a, b)	The date and time in time zone b when date d is for time zone a
NEXT_DAY (d, day)	Date of the first day of the week after d
SYSDATE	Current date and time
GREATEST (d1, d2, ..., dn)	Latest of the given dates
LEAST (d1, d2, ..., dn)	Earliest of the given dates

For example

```
SELECT SYSDATE();
```

Will result in

```
2019-06-27 10:02:39
```

2.10 NULL Values

- In SQL NULL is an important value. NULL is nothing but a missing value. But it has three different interpretations -
 - 1) **Unknown Value** : If student's age is not known then the NULL represents the unknown value.
 - 2) **Unavailable Value** : If a persons' phone number is not available then it can be represented by NULL value in the database
 - 3) **Not applicable Attribute** : If some value is not applicable to current database systems then it can be represented by NULL. For instance - Parent's designation is not applicable in student database system.
- When a record with NULL in one of its attributes is involved in a comparison operation, result is UNKNOWN.

- SQL uses a **three-valued** logic with values TRUE, FALSE, and UNKNOWN instead of the standard two-valued (Boolean) logic with values TRUE or FALSE. Using the logical operators AND, OR and NOT the result of these three values is represented in the following table -

AND	TRUE	FALSE	UNKNOWN
TRUE	TRUE	FALSE	UNKNOWN
FALSE	FALSE	FALSE	FALSE
UNKNOWN	UNKNOWN	FALSE	UNKNOWN

OR	TRUE	FALSE	UNKNOWN
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	UNKNOWN
UNKNOWN	TRUE	UNKNOWN	UNKNOWN

NOT		
TRUE	FALSE	
FALSE	TRUE	
UNKNOWN	UNKNOWN	

- For example - The result of FALSE AND TRUE is FALSE
- SQL allows queries that check whether an attribute value is NULL. For that, instead of using =, <> (not equal to), it compares attribute value to NULL using the operators IS or IS NOT.
- For instance - Retrieve names of all students who do not opt for any sports.

```
SELECT FName, LName
FROM STUDENT
WHERE sports_type IS NULL
```

2.11 EXISTS, NOT EXISTS and UNIQUE

- The **EXISTS** operator is used to test for the existence of any record in a subquery. The **EXISTS** operator returns true if the subquery returns one or more records.

- Syntax**

```
SELECT column_name(s)
FROM table_name
WHERE EXISTS
(SELECT column_name FROM table_name WHERE condition);
```

- Example - Consider two tables namely Student and Student_Score table as follows -

Student Table

RollNo	FName	LName
1.	Anil	Kumble
2.	Virat	Kohli
3.	Suresh	Raina

Student_Score Table

Subject	TestNo	Grade	RollNo
Maths	1	10	1
Maths	2	9.5	1
Maths	3	9.75	1
Science	4	9.5	1
Science	5	8.5	1
Maths	6	9.25	2
Maths	7	9.8	2
Science	8	7	2

Let's say we want to get all students that have received a 10 grade in the subject 'Maths'.

The SQL query using EXISTS can be written as follows –

```
SELECT
    RollNo, FName, LName
FROM
    Student
WHERE EXISTS (
    SELECT Student_Score.RollNo
    FROM
        Student_Score
    WHERE
        Student_Score.RollNo = Student.RollNo AND
        Student_Score.grade = 10 AND
        Student_Score.Subject = 'Maths'
)
```

After running the above query the result will be -

RollNo	FName	LName
1.	Anil	Kumble

The outer query selects the student row columns we are interested in returning to the client. However, the WHERE clause is using the EXISTS operator with an associated inner subquery.

- The EXISTS operator returns true if the subquery returns at least one record and false if no row is selected.
- The **NOT EXISTS** operator returns true if the underlying subquery returns no record. However, if a single record is matched by the inner subquery, the NOT EXISTS operator will return false, and the subquery execution can be stopped.
- The **UNIQUE(Q)** function returns TRUE if there are no duplicate tuples in the result of query Q.
- **Syntax**

```
SELECT UNIQUE column_name1, column_name2, ..., column_namen
FROM table_name;
```

- **Example** - Suppose there is a Student table as follows –

Student

RollNo	Name	City	Age
1	AAA	Pune	19
2	BBB	Mumbai	19
3	CCC	Chennai	20
4	DDD	Delhi	20

If we execute following query

```
SELECT Age FROM Student;
```

Then the result will be

Age
19
19
20
20

Now if we execute following query

```
SELECT UNIQUE Age FROM Student;
```

The result will be

Age
19
20

2.12 Defining Constraints

- We can specify rules for data in a table.
- When the table is created at that time we can define the constraints.
- The constraint can be column level i.e. we can impose constraint on the column and table level i.e we can impose constraint on the entire table.
- There are various types of constraints that can be defined are as follows -

(1) Primary key : The primary key constraint is defined to uniquely identify the records from the table.

The primary key must contain unique values. Hence database designer should choose primary key very carefully.

For example

Consider that we have to create a **perons_details** table with AdharNo, FirstName, MiddleName, LastName, Address and City.

Now making AdharNo as a primary key is helpful here as using this field it becomes easy to identify the records correctly.

The result will be

```
CREATE TABLE person_details (  
  AadharNo int,  
  FirstName VARCHAR(20),  
  MiddleName VARCHAR(20),  
  LastName VARCHAR(20),  
  Address VARCHAR(30),  
  City VARCHAR(10),  
  PRIMARY KEY(AadharNo)  
);
```

We can create a composite key as a primary key using CONSTRAINT keyword. For example

```
CREATE TABLE person_details (  
  AadharNo int NOT NULL,  
  FirstName VARCHAR(20),  
  MiddleName VARCHAR(20),  
  LastName VARCHAR(20) NOT NULL,  
  Address VARCHAR(30),  
  City VARCHAR(10),  
  CONSTRAINT PK_person_details PRIMARY KEY(AadharNo, LastName)  
);
```

(2) Foreign Key

- Foreign key is used to link two tables.
- Foreign key for one table is actually a primary key of another table.
- The table containing foreign key is called child table and the table containing candidate primary key is called parent key.
- Consider

Employee Table

EmpID	LastName	FirstName	Age
1	Khanna	Rajesh	30
2	Joshi	Sharman	23
3	Kapoor	Tushar	20

Dept Table :

DeptID	DeptName	EmpID
1	Accounts	3
2	Production	3
3	Sales	2
4	Purchase	1

- Notice that the "EmpID" column in the "Dept" table points to the "EmpID" column in the "Employee" table.
- The "EmpID" column in the "Employee" table is the PRIMARY KEY in the "Employee" table.
- The "EmpID" column in the "Dept" table is a FOREIGN KEY in the "Dept" table.
- The FOREIGN KEY constraint is used to prevent actions that would destroy links between tables.
- The FOREIGN KEY constraint also prevents invalid data from being inserted into the foreign key column, because it has to be one of the values contained in the table it points to.
- The purpose of the foreign key constraint is to enforce referential integrity but there are also performance benefits to be had by including them in your database design.

The table Dept can be created as follows with foreign key constraint.

```
CREATE TABLE DEPT (  
    DeptID      int  
    DeptName VARCHAR(20),  
    EmpID int,  
    PRIMARY KEY(DeptID),  
    FOREIGN KEY(EmpID) REFERENCES EMPLOYEE(EmpID)  
);
```

(3) Unique

Unique constraint is used to prevent same values in a column. In the EMPLOYEE table, for example, you might want to prevent two or more employees from having an identical **designation**. Then in that case we must use unique constraint.

We can set the constraint as unique at the time of creation of table, or if the table is already created and we want to add the unique constraint then we can use ALTER command.

For example –

```
CREATE TABLE EMPLOYEE(  
    EmpID INT NOT NULL,  
    Name VARCHAR (20) NOT NULL,  
    Designation VARCHAR(20) NOT NULL UNIQUE,  
    Salary DECIMAL (12, 2),  
    PRIMARY KEY (EmpID)  
);
```

If table is already created then also we can add the unique constraint as follows –

```
ALTER TABLE EMPLOYEE  
MODIFY Designation VARCHAR(20) NOT NULL UNIQUE;
```

(4) NOT NULL

- By default the column can have NULL values.
- NULL means unknown values.
- We can set the column values as non NULL by using the constraint NOT NULL.
- For example –

```
CREATE TABLE EMPLOYEE(  
    EmpID INT NOT NULL,  
    Name VARCHAR (20) NOT NULL,  
    Designation VARCHAR(20) NOT NULL,  
    Salary DECIMAL (12, 2) NOT NULL,  
    PRIMARY KEY (EmpID)  
);
```

(5) CHECK

The CHECK constraint is used to limit the value range that can be placed in a column.

For example

```
CREATE TABLE parts (  
    part_no int PRIMARY KEY,  
    description VARCHAR(40),  
    price DECIMAL(10 , 2 ) NOT NULL CHECK(cost > 0)  
);
```

(6) IN operator

The IN operator is just similar to OR operator.

It allows to specify multiple values in WHERE clause.

Syntax

```
SELECT col1,col2,...  
FROM table_name  
WHERE column-name IN (value1, value2, ...);
```

Example

Consider following table

Employee

empID	empName	Salary	DeptID
1	AAA	1000	D101
2	BBB	2000	D102
3	CCC	3000	D103
4	DDD	4000	D104
5	EEE	5000	D105

```
SELECT * FROM Employee
WHERE empID IN (1, 3);
```

The result will be

empID	empName	Salary	DeptID
1	AAA	1000	D101
2	BBB	2000	D102
3	CCC	3000	D103

2.13 Renaming Attributes

The SQL **AS** is used to assign temporarily a new name to a table column or table(relation) itself. One reason to rename a relation is to replace a long relation name with a shortened version that is more convenient to use elsewhere in the query. For example - "Find the names of students and isbn of book who reserve the books".

Student

sid	sname	age
1	Ram	21
2	Shyam	18
3	Seeta	16
4	Geeta	23

Reserve

sid	isbn	day
1	005	07-07-18
2	007	03-03-18
3	009	05-05-18

```
Select S.sname,R.isbn
From Student as S, Reserve as R
Where S.sid=R.sid
```

In above case we could shorten the names of tables Student and Reserve as S and R respectively.

Another reason to rename a relation is a case where we wish to compare tuples in the same relation. We then need to take the **Cartesian product** of a relation with itself. For example –

If the query is - Find the names of students who reserve the book of isbn 005. Then the SQL statement will be -

```
Select S.sname,R.isbn
From Student as S, Reserve as R
Where S.sid=R.sid and S.isbn=005
```

2.14 Tuple Variables

Tuple variables can be used in SQL and are defined in the FROM clause.

For example –

```
SELECT DISTINCT cname, C.courseID
FROM Student AS S, Course AS C
WHERE S.courseID = C.courseID
```

Note that we have used the tuple variables as S.courseID and C.courseID. The keyword AS is optional here. Thus tuple variables can be used throughout the expression in SQL.

2.15 Schema Change Statements

SPPU : Nov.-18,(End Sem), Marks 5

Schema can be changed by adding or dropping tables, attributes and constraints. These commands are also known as **schema evolution commands**.

There are two commands that change the schema and those are DROP and ALTER.

Let us discuss these commands in detail

2.15.1 The DROP Command

- The DROP command is used to remove the object (table, domains and constraints) from the database. There are two options for the DROP command - CASCADE and RESTRICT.
- To use the RESTRICT option, the user must first individually drop each element in the schema, then drop the schema itself. That means, the schema is dropped only if it has no elements in it, otherwise the DROP command can not be executed.
- Otherwise to remove completely some database schema CASCADE option is chosen. For example – to remove the **Student_database**

```
DROP SCHEMA Student_database CASCADE;
```

- If the table is to be deleted then the SQL command would be -

```
DROP TABLE Student CASCADE;
```

- The DROP TABLE command not only deletes all the records in the table if successful, but also removes the table definition from the catalog. If it is desired to delete only the records but to leave the table definition for future use, then the DELETE command. For example -
- The following SQL statement deletes all rows in the "Students" table, without deleting the table :

```
DELETE FROM Students;
```

2.15.2 The ALTER Command

There are SQL commands for alteration of table. That means we can add new column or delete some column from the table using these alteration commands.

Syntax for Adding columns

```
ALTER TABLE table_name  
ADD column_name datatype;
```

Example

Consider following table

AadharNo	FirstName	MiddleName	LastName	Address	City
111	AAA	BBB	CCC	M.G. Road	Pune
222	DDD	EEE	FFF	Shivaji nagar	Pune
333	GGG	HHH	III	Chandani chowk	Delhi
444	JJJ	KKK	LLL	Viman nagar	Mumbai

If we execute following command

```
ALTER TABLE Customers  
ADD Email varchar(30);
```

Then the result will be as follows -

AadharNo	FirstName	MiddleName	LastName	Address	City	Email
111	AAA	BBB	CCC	M.G. road	Pune	NULL
222	DDD	EEE	FFF	Shivaji nagar	Pune	NULL
333	GGG	HHH	III	Chandani chowk	Delhi	NULL
444	JJJ	KKK	LLL	Viman nagar	Mumbai	NULL

Syntax for Deleting columns

```
ALTER TABLE table_name
DROP COLUMN column_name;
```

Example

Consider following table -

AadharNo	FirstName	MiddleName	LastName	Address	City
111	AAA	BBB	CCC	M.G. road	Pune
222	DDD	EEE	FFF	Shivaji nagar	Pune
333	GGG	HHH	III	Chandani chowk	Delhi
444	JJJ	KKK	LLL	Viman nagar	Mumbai

If we execute following command

```
ALTER TABLE Customers
DROP COLUMN Address;
```

Then the result will be as follows -

AadharNo	FirstName	MiddleName	LastName	City
111	AAA	BBB	CCC	Pune
222	DDD	EEE	FFF	Pune
333	GGG	HHH	III	Delhi
444	JJJ	KKK	LLL	Mumbai

2.16 Indexes**SPPU : Aug. 17, In Sem, Dec. 18, May 19, End Sem, Marks 5**

Index is a pointer to the data in the database table. These are also called as **lookup tables**. With the help of indexing data retrieval becomes fast and efficient. The concept of index is just similar to the index at the back of the book which contains the keywords. Using these keywords it is easy to locate the desired record quickly from the database table.

1) Creating an Index**Syntax for creating an Index**

```
CREATE INDEX index_name ON table_name;
```

Example

Consider following Book table

isbn	bname	Author
005	DBMS	XYZ
006	OS	PQR
007	DAA	ABC

We can create an index named **idx_isbn** using the field **isbn** of the **Book** table.

```
CREATE INDEX idx_isbn  
ON Book(isbn);
```

Creating Index using UNIQUE**Syntax**

```
CREATE UNIQUE INDEX index_name  
ON table_name (column1, column2, ...);
```

2) Creating Index on Multiple Columns

We can create index using more than one columns of the table. For example – For the above given Book table we can create an index as

```
CREATE INDEX idx_isbn  
ON Book(bname, Author);
```

3) Dropping the Index

The DROP INDEX statement is used to delete an index in a table.

The syntax is

```
DROP INDEX index_name
```


Example 2.16.1 *It is easy to create index on all attributes of any relation, why it is not recommended to create index on all attributes ?*

SPPU : Dec 18, May 19, End Sem, Marks 5

Solution : The index is used to access the desired records efficiently. If we create index on a single attribute then the record can be accessed quickly. As every index requires some disk space and if we create index on all the attributes of any relation then it such index will require more memory.

Secondly, every time when we add or update the record then we have to recalculate the indexes and it will take a lot of time and will cause to perform less efficiently.

Hence it is not recommended to create index on all attributes

Review Question

1. What is index created on table column ? How performance of SELECT query is improved if index is created on table ?

SPPU : Aug 17, In Sem, Marks 5

2.17 Aggregate Functions

SPPU : Nov.-17, (End Sem), Marks 2

- An aggregate function allows you to perform a calculation on a set of values to return a single scalar value.
- SQL offers five built-in aggregate functions :
 1. Average : avg
 2. Minimum : min
 3. Maximum : max
 4. Total : sum
 5. Count :
- The aggregate functions that accept an expression parameter can be modified by the keywords DISTINCT or ALL. If neither is specified, the result is the same as if ALL were specified.

DISTINCT	Modifies the expression to include only distinct values that are not NULL
ALL	Includes all rows where expression is not NULL

- Syntax of all the Aggregate Functions

```
AVG( [ DISTINCT | ALL ] expression)
COUNT(*)
COUNT( [ DISTINCT | ALL ] expression )
MAX( [ DISTINCT | ALL ] expression)
MIN( [ DISTINCT | ALL ] expression)
SUM( [ DISTINCT | ALL ] expression)
```

- The **avg** function is used to compute average value. For example – To compute average marks of the students we can use

SQL Statement
 SELECT AVG(marks)
 FROM Students

- The **Count** function is used to count the total number of values in the specified field. It works on both numeric and non-numeric data type. COUNT (*) is a special implementation of the COUNT function that returns the count of all the rows in a specified table. COUNT (*) also considers Nulls and duplicates. For example Consider following table

Test	
id	value
11	100
22	200
33	300
NULL	400

SQL Statement

```
SELECT COUNT(*)  
FROM Test  
Output  
4  
SELECT COUNT(ALL id)  
FROM Test  
Output  
3
```

- The **min** function is used to get the minimum value from the specified column. For example – Consider the above created **Test table**

```
SQL Statement  
SELECT Min(value)  
FROM Test  
Output  
100
```

- The max function is used to get the maximum value from the specified column. For example - Consider the above created **Test table**

```
SQL Statement  
SELECT Max(value)  
FROM Test  
Output  
400
```

- The sum function is used to get total sum value from the specified column. For example - Consider the above created **Test table**

```
SQL Statement  
SELECT sum(value)  
FROM Test
```

Output

1000

Example 2.17.1 Consider, the following database,*Student(RollNo, Name, Address)**Subject(Sub_code, Sub_Name)**Marks (Roll_no, Sub_code, Marks)**Write following queries in SQL.**Find average marks of each student, along with the name of Student***SPPU : Nov.-17, (End Sem), Marks 2****Solution:**

```
SELECT Name, AVG(Marks)
FROM Student,Marks
WHERE Student.Roll_No=Marks.Roll_No
```

2.18 Set Operations

Set is a collection of elements on which union, intersection and difference operations can be performed.

1) Union : To use this UNION clause, each SELECT statement must have

- i) The same number of columns selected
- ii) The same number of column expressions
- iii) The same data type and
- iv) Have them in the same order

This clause is used to **combine two tables** using UNION operator. It replaces the OR operator in the query. The union operator **eliminates duplicate** while the union all query will retain the duplicates.

Syntax :

The basic syntax of a UNION clause is as follows -

```
SELECT column1 [, column2 ]
FROM table1 [, table2 ]
[WHERE condition]
UNION
SELECT column1 [, column2 ]
FROM table1 [, table2 ]
[WHERE condition]
```

Here, the given condition could be any given expression based on your requirement.

Example : Find the names of the students who have reserved the 'DBMS' book or 'OS' Book.

The query can then be written by considering the Student, Reserve and Book table as

```
SELECT S.sname
FROM Student S, Reserve R, Book B
WHERE S.sid=R.sid AND R.isbn=B.isbn AND B.bname='DBMS'
UNION
SELECT S.sname
FROM Student S, Reserve R, Book B
WHERE S.sid=R.sid AND R.isbn=B.isbn AND B.bname='OS'
```

- 2) **Intersect** : The common entries between the two tables can be represented with the help of Intersect operator. It replaces the AND operator in the query.

Syntax :

The basic syntax of a INTERSECT clause is as follows-

```
SELECT column1 [, column2 ]
FROM table1 [, table2 ]
[WHERE condition]
INTERSECT
SELECT column1 [, column2 ]
FROM table1 [, table2 ]
[WHERE condition]
```

Example : Find the students who have reserved both the 'DBMS' book and 'OS' Book

The query can then be written by considering the Student, Reserve and Book table as

```
SELECT S.sid, S.sname
FROM Student S, Reserve R, Book B
WHERE S.sid=R.sid AND R.isbn=B.isbn AND B.bname='DBMS'
INTERSECT
SELECT S.sname
FROM Student S, Reserve R, Book B
WHERE S.sid=R.sid AND R.isbn=B.isbn AND B.bname='OS'
```

- 3) **Except** : The EXCEPT clause is used to represent the set-difference in the query. This query is used to represent the entries that are present in one table and not in other.

Syntax :

The basic syntax of a EXCEPT clause is as follows-

```
SELECT column1 [, column2 ]
FROM table1 [, table2 ]
[WHERE condition]
EXCEPT
SELECT column1 [, column2 ]
FROM table1 [, table2 ]
[WHERE condition]
```

Example : Find the students who have reserved both the 'DBMS' book but not reserved 'OS' Book

The query can then be written by considering the Student, Reserve and Book table as

```
SELECT S.sid, S.sname
FROM Student S, Reserve R, Book B
WHERE S.sid=R.sid AND R.isbn=B.isbn AND B.bname='DBMS'
EXCEPT
SELECT S.sname
FROM Student S, Reserve R, Book B
WHERE S.sid=R.sid AND R.isbn=B.isbn AND B.bname='OS'
```

2.19 Nested Queries

In nested queries, a **query is written inside a query**. The result of inner query is used in execution of outer query.

There are two types of nested queries :

(i) Independent Query (ii) Corelated Query

i) Independent Query :

- In independent nested queries, query execution starts from innermost query to outermost queries.
- The execution of inner query is independent of outer query, but the result of inner query is used in execution of outer query.
- Various operators like IN, NOT IN, ANY, ALL etc are used in writing independent nested queries.
- For example - Consider three tables namely **Student**, **City** and **Student_City** as follows -

Student		
sid	sname	phone
1	Ram	1111
2	Shyam	2222
3	Seeta	3333
4	Geeta	4444

City	
cid	cname
101	Pune
102	Mumbai
103	Chennai

Student_City	
sid	cid
1	101
1	103
2	101
3	102
4	102
4	103

- **Example 1** - If we want to find out **sid** who live in city 'Pune' or 'Chennai'. We can then write independent nested query using IN operator. Here we can use the IN operator allows you to specify multiple values in a WHERE clause. The IN operator is a shorthand for multiple OR conditions.

Step 1 : Find cid for cname='Pune' or 'Chennai'. The query will be

```
SELECT cid
FROM City
WHERE cname='Pune' or 'Chennai'
```

Step 2 : Using cid obtained in step 1 we can find the sid. The query will be

```
SELECT sid
FROM Student_City
WHERE cid IN
(
  SELECT cid
  FROM City
  WHERE cname='Pune' or cname='Chennai'
)
```

The inner query will return a set with members 101 and 103 and outer query will return those **sid** for which **cid** is equal to any member of set (101 and 103 in this case). So, it will return 1, 2 and 4.

Example 2 : If we want to find out **sname** who live in city 'Pune' or 'Chennai'.

```
SELECT sname
FROM Student
WHERE sid IN
( SELECT sid
  FROM Student_City
```

```
WHERE cid IN
( SELECT cid
  FROM City
  WHERE cname='Pune' or cname='Chennai'
)
)
```

ii) Co-related Query :

In co-related nested queries, the output of inner query depends on the row which is being currently executed in outer query. For example

If we want to find out sname of Student who live in city with cid as 101, it can be done with the help of co-related nested query as :

```
SELECT sname
FROM Student S
WHERE EXISTS
( SELECT *
  FROM Student_City SC
  WHERE S.sid=SC.sid and SC.cid=101
)
```

Here for each row of **Student S**, it will find the rows from **Student_City** where S.sid = SC.sid and SC.cid=101.

If for a **sid** from **Student S**, atleast a row exists in **Student_City SC** with **cid**=101, then inner query will return true and corresponding **sid** will be returned as output.

2.20 Join Operation

SPPU : Nov.-17,19, May-19, Marks 6

The SQL Joins clause is used to combine records from two or more tables in a database. A JOIN is a means for combining fields from two tables by using values common to each.

Various types of join operations are –

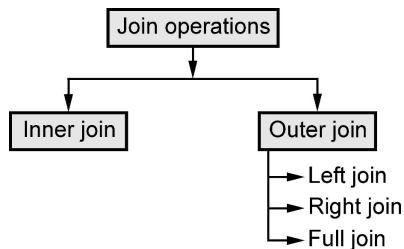


Fig. 2.20.1 Types of join operations

Example : Consider two tables for using the joins in SQL. Note that **cid** is common column in following tables.

Student

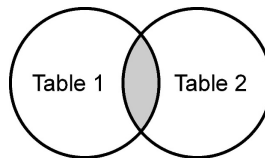
sid	cid	sname
1	101	Ram
2	101	Shyam
3	102	Seeta
4	NULL	Geeta

Reserve

cid	cname
101	Pune
102	Mumbai
103	Chennai

1) Inner Join :

- The most important and frequently used of the joins is the INNER JOIN. They are also known as an EQUIJOIN.
- The INNER JOIN creates a new result table by combining column values of two tables (Table1 and Table2) based upon the join-predicate.
- The query compares each row of Table 1 with each row of Table 2 to find all pairs of rows which satisfy the join-predicate.
- When the join-predicate is satisfied, column values for each matched pair of rows of A and B are combined into a result row. It can be represented as :



- **Syntax :** The basic syntax of the INNER JOIN is as follows.

```
SELECT Table1.column1, Table2.column2...
FROM Table1
INNER JOIN Table2
ON Table1.common_field = Table2.common_field;
```

- **Example :** For above given two tables namely Student and City, we can apply inner join. It will return the record that are matching in both tables using the common column cid.

The query will be

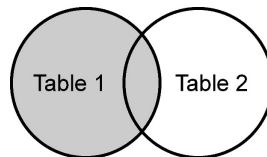
```
SELECT *
FROM Student Inner Join City on Student.cid=City.cid
```


The result will be

sid	cid	sname	cid	cname
1	101	Ram	101	Pune
2	101	Shyam	101	Pune
3	102	Seeta	102	Mumbai

2) Left Join(Outer Join) :

- The SQL LEFT JOIN returns all rows from the left table, even if there are no matches in the right table. This means that if the ON clause matches 0 (zero) records in the right table; the join will still return a row in the result, but with NULL in each column from the right table.
- This means that a left join returns all the values from the left table, plus matched values from the right table or NULL in case of no matching join predicate.
- It can be represented as –



- **Syntax :** The basic syntax of a LEFT JOIN is as follows.

```
SELECT
SELECT Table1.column1, Table2.column2...
FROM Table1
LEFT JOIN Table2
ON Table1.common_field = Table2.common_field;
```

- **Example :** For above given two tables namely Student and City, we can apply Left join. It will Return all records from the left table, and the matched records from the right table using the common column cid. The query will be

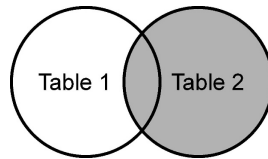
```
SELECT *
FROM Student Left Join City on Student.cid=City.cid
```

The result will be

sid	cid	sname	cid	cname
1	101	Ram	101	Pune
2	101	Shyam	101	Pune
3	102	Seeta	102	Mumbai
4	NULL	Geeta	NULL	NULL

3) Right Join(Outer Join) :

- The SQL RIGHT JOIN returns all rows from the right table, even if there are no matches in the left table.
- This means that if the ON clause matches 0 (zero) records in the left table; the join will still return a row in the result, but with NULL in each column from the left table.
- This means that a right join returns all the values from the right table, plus matched values from the left table or NULL in case of no matching join predicate.
- It can be represented as follows :



- **Syntax :** The basic syntax of a RIGHT JOIN is as follow -

```
SELECT Table1.column1, Table2.column2...  
FROM Table1  
RIGHT JOIN Table2  
ON Table1.common_field = Table2.common_field;
```

- **Example :** For above given two tables namely Student and City, we can apply Right join. It will return all records from the right table, and the matched records from the left table using the common column cid. The query will be

```
SELECT *  
FROM Student Right Join City on Student.cid=City.cid
```

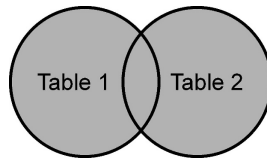
The result will be –

sid	cid	sname	cid	cname
1	101	Ram	101	Pune
2	101	Shyam	101	Pune
3	102	Seeta	102	Mumbai
NULL	NULL	NULL	103	Chennai

4) Full Join (Outer Join) :

- The SQL FULL JOIN combines the results of both left and right outer joins.
- The joined table will contain all records from both the tables and fill in NULLs for missing matches on either side.

- It can be represented as,



- **Syntax :** The basic syntax of a FULL JOIN is as follows :

```
SELECT Table1.column1, Table2.column2...
FROM Table1 FULL JOIN Table2 ON Table1.common_field = Table2.common_field;
```

The result will be -

- **Example :** For above given two tables namely **Student** and **City**, we can apply full join. It will return rows when there is a match in one of the tables using the common column cid. The query will be -

```
SELECT *
FROM Student Full Join City on Student.cid=City.cid
```

The result will be -

sid	cid	sname	cid	cname
1	101	Ram	101	Pune
2	101	Shyam	101	Pune
3	102	Seeta	102	Mumbai
4	NULL	Geeta	NULL	NULL
NULL	NULL	NULL	103	Chennai

2.21 Views

SPPU : July-18, Oct.-18, (In Sem), Nov.-19, Marks 8

- Views in SQL are kind of **virtual tables**.
- A view also has rows and columns as they are in a real table in the database.
- We can create a view by selecting fields from one or more tables present in the database.
- A view can either have all the rows of a table or specific rows based on certain condition.

Creating View

We can create a view using CREATE VIEW statement. The syntax is

```
CREATE VIEW name_of_view AS
SELECT column1,column2,...
FROM table_name1,table_name2,...
WHERE condition;
```

Example

- i) **Creating a view using single table :** Consider table Employee and create a view EmployeeDetails whose Salary is < 10000

EmpID	EName	Salary
101	Archana	20000
102	Madhura	5000
103	Poonam	8000
104	Sharda	15000
105	Monika	7000

```
CREATE VIEW EmployeeDetails(EmpID, EName) AS
SELECT E.EmpID, E.EName
FROM Employee E
WHERE E.Salary > 10000
```

The **Output** will be -

EmpID	EName
102	Madhura
103	Poonam
105	Monika

- ii) **Creating a view from multiple table :** In this example we will create a view from two tables Employee and Department.

Employee

EmpID	EName	Salary
101	Archana	20000
102	Madhura	5000
103	Poonam	8000
104	Sharda	15000
105	Monika	7000

Department

EmpID	DName
101	Accounts
104	Sales
105	Sales

Now we need to create a view named Employee_dept_Details in which the names and salary of employees belonging to Sales department is displayed. The SQL statement will then be –

```
CREATE VIEW Employee_dept_Details(EName,Salary) AS
SELECT E.EName,E.Salary
FROM Employee E, Department D
WHERE E.EmpID=D.EmpID AND D.DName='Sales'
```

The output will be

EName	Salary
Sharda	15000
Monika	7000

View Update

- The SQL UPDATE VIEW command can be used to modify the data of a view.
- All views are not updatable. So, UPDATE command is not applicable to all views.
- An updatable view is one which allows performing a UPDATE command on itself without affecting any other table.

- **Syntax for updating view**

```
UPDATE < view_name >  
SET <column1>=<value1>,<column2>=<value2>,...  
WHERE <condition>;
```

- **Example**

```
UPDATE view Employee_dept_Details  
SET Salary=1000  
WHERE EName='Monika';  
This view can be viewed by using following query  
SELECT * FROM Employee_dept_Details;
```

Rules for updating the views

The view can be updated in following conditions :

- (1) The view can be defined based on one and only one table.
- (2) The SELECT statement should not have DISTINCT keyword.
- (3) The view should not have all NOT NULL values.
- (4) The view should not be created from nested and complex queries.
- (5) The SELECT statement which is used to create the view should not include GROUP BY clause or ORDER BY clause.
- (6) The view should not have any field made out of aggregate functions.
- (7) Any selected output fields of the view must not use constants, strings or value expressions.

Dropping View

For deleting a view, the DROP command is used.

Syntax

```
DROP VIEW view_name;
```

Example

```
DROP VIEW Employee_dept_Details;
```

Difference between View and Table

- Table is a database object that contains the data in row and column form. View is also database object and it is virtual table which is built on the top of the other tables.
- The table contains the data while view does not hold data itself.
- A table is designed with a limited number of columns and an unlimited number of rows while a view is designed as a virtual table that is extracted from a database.

Example 2.21.1 Consider following company database :

EMP(Name, SSn, Salary, Supersn, dno)

DEPT (dnum, dname, mgrssn)

DEPT_LOC (dnum, dlocation)

PROJECT(Pname, Pnumber, Plocation, dnum)

WORKS_ON(Essn, dept_name, sex)

Write SQL queries for the following:

- i) Retrieve the names of all employees who work in the department that has the employee with the highest salary among all employees.
- ii) Retrieve the names of employees who make atleast 10000 more than the employee who is paid the least in the company.
- iii) A view that has the employee name, supervisor name and employee salary for each employee who works in the Research' department.
- iv) A view that has the project name, controlling department name, number of employees and total hours worked per week on the project for each project with more than one employee working on it.

Solution : (i)

```
SELECT Name FROM EMP WHERE dno =
      ( SELECT dno FROM EMP WHERE Salary =
        ( SELECT MAX(Salary) FROM EMP ) )
```

(ii)

```
SELECT Name FROM EMP WHERE Salary >= 10000 +
      ( SELECT MIN(Salary) FROM EMP )
```

(iii)

```
CREATE VIEW RESEARCH_EMPLOYEE_INFO (Lname, Fname, Supervisor, Salary) AS
SELECT E.Lname, E.Fname, S.Lname, E.Salary FROM
EMPLOYEE E, EMPLOYEE S, DEPARTMENT D
WHERE Dname = 'Research' AND
D.Dnumber = E.Dno AND E.super_ssn = S.ssn;
```

(iv)

```
CREATE VIEW PROJECT_INFO AS
SELECT Pname, Dname, COUNT(WO.Essn), SUM(WO.Hours)
FROM PROJECT P, DEPARTMENT D, WORKS_ON WO
WHERE
P.Dnum = D.Dnumber AND
P.Pnumber = WO.Pno GROUP BY Pno
HAVING COUNT(WO.Essn) > 1
```

Review Question

1. What is the use of database view ? What are the mandatory requirements of updating the values in view ? Explain with example.

SPPU : Oct.-18, (In Sem), Marks 5**2.22 Examples Based on SQL**

Example 2.22.1 For the following database, identify primary key and foreign key where-ever applicable and solve the given queries in SQL.

Item (ino, description, unit_price)

Supplier (sno, sname, address)

Supplied (sno, ino, sdate, qty, per_unit_discount)

1. Find supplier name for the suppliers who supply every item.
2. Find distinct item names for the items supplied with total discount > 500.
3. Pair of supplier names supplying same items on same dates.

Solution :

- The primary key for Item table is **ino**.
- The primary key for supplier table is **sno**
- The primary key for supplied table is **(sno, ino)**

1) Find supplier name for the suppliers who supply every item.

```
SELECT S.sname
FROM Supplier S
WHERE NOT EXISTS (
    ( SELECT * FROM Item I
      WHERE NOT EXISTS
        ( SELECT *
          FROM Supplied SP
          WHERE SP.sno = S.sno
            AND SP.ino=I.ino )
    )
)
```

2) Find distinct item names for the items supplied with total discount > 500.

```
SELECT DISTINCT description
FROM Item, Supplied
WHERE Item.ino=Supplied.ino
AND Supplied.per_unit_discount >500
```

3) Pair of supplier names supplying same items on same dates.

```
SELECT sname
FROM Supplier
WHERE Supplied.sno=Supplier.sno
AND
Item.ino=Supplied.ino
```


Example 2.22.2 Write SQL statements for the following (any five)

Consider the following database

pilot (pid, pname)

flight (fid, ftype, capacity)

route (pid, fid, from_city, to_city)

i) List the details of flights having capacity more than 300.

ii) List the flights between 'Surat' and 'Mumbai'.

iii) List the names of the pilots who fly from 'Pune'.

iv) List the route on which, pilot named 'Mr Kapoor' flies

v) List the pilots whose names, starts with letter 'A' %' but does not end with letter 'A'.

vi) List the name of pilots who fly 'boing 737' type of flights.

Solution :

(i) List the details of flights having capacity more than 300.

```
SELECT * FROM flight
WHERE capacity > 300
```

(ii) List the flights between 'Surat' and 'Mumbai'.

```
SELECT fid
FROM flight, route
WHERE
flight.fid = route.fid AND
route.from_city = 'Surat' AND route.to_city = 'Mumbai';
```

(iii) List the names of the pilots who fly from 'Pune'.

```
SELECT pname
FROM pilot, route
WHERE pilot.pid = route.pid
AND route.from_city = 'Pune';
```

(iv) List the route on which, pilot named 'Mr Kapoor' flies.

```
SELECT from_city, to_city
FROM route, pilot
WHERE pilot.pid = route.pid
AND pilot.pname = 'Mr.Kapoor';
```

(v) List the pilots whose names, starts with letter 'A' %' but does not end with letter 'A'.

```
SELECT pname
FROM pilot
WHERE pname LIKE 'A%' MINUS
SELECT pname FROM pilot
WHERE pname LIKE '%A';
```

(vi) List the name of pilots who fly 'boing 737' type of flights.

```
SELECT pname
FROM pilot, flight, route
WHERE flight.ftype = 'boing 737' AND
pilot.pid = route.pid AND
route.fid = flight.fid;
```

Example 2.22.3 Consider the relation Database.

Person (SSN, Name, city)

Car (License_no, year, Model, SSN)

Accident(drive_no, SSN, license_no, accidentyear, damage_Amt)

Query :

1. Find out total no of cars that had accident in 1988.
2. Find the Name of driver who did not have an accident in 'Delhi'.
3. Find the car, who don't have total damage of more than ₹1000.
4. Find the cars sold in 2006 and whose owner are from 'Vadodara'
5. How many different models of car are used by 'Mr.abc'
6. Find the lucky persons who have not met any accident yet.

Solution :**1. Find out total no of cars that had accident in 1988.**

```
SELECT count(License_no)
FROM Car, Accident
WHERE Accident.accidentyear=1988
```

2. Find the Name of driver who did not have an accident in 'Delhi'.

```
SELECT Name
FROM Person,Accident
WHERE Person.SSN=Accident.SSN AND Person.city<> 'Delhi'
```

3. Find the car, who don't have total damage of more than ` 1000.

```
SELECT License_no
FROM Car, Accident
WHERE Accident.damage_amt<1000 AND
Car.License_no=Accident.licence_no;
```

4. Find the cars sold in 2006 and whose owner are from 'Vadodara'

```
SELECT license_no
FROM Car, Person
WHERE Car.SSN = Person. SSN AND
Car.year = 2006 and Person.city = 'Vadodara';
```

5. How many different models of car are used by 'Mr.abc'

```
SELECT count(model)
FROM Car, Person
WHERE Car.SSN = Person. SSN AND
Person.Name = 'Mr.abc';
```

6. Find the lucky persons who have not met any accident yet.

```
SELECT Name
FROM Person
WHERE person.SSN NOT IN (SELECT SSN FROM Accident);
```

Example 2.22.4 We have following relations :

Supplier (S#, sname, status, city)

Parts (P#, pname, color, weight, city)

SP(S#, P#, quantity)

Answer the following queries in SQL,

i) Find name of supplier for city = 'Delhi'.

ii) Find suppliers whose name start with 'AB'.

iii) Find all suppliers whose status is 10, 20 or 30.

iv) Find total number of city of all suppliers.

v) Find s# of supplier who supplies 'red' part.

vi) Count number of supplier who supplies 'red' part.

vii) Sort the supplier table by sname.

Solution : i) Find name of supplier for city = 'Delhi'.

```
SELECT sname
FROM Supplier
WHERE city='Delhi'
```

ii) Find suppliers whose name start with 'AB'.

```
SELECT sname
FROM Supplier
WHERE sname='AB%'
```

iii) Find all suppliers whose status is 10, 20 or 30.

```
SELECT sname
FROM Supplier
WHERE status BETWEEN 10 AND 30
```

iv) Find total number of city of all suppliers.

```
SELECT count(*) FROM (SELECT DISTINCT CITY FROM Supplier);
```

v) Find s# of supplier who supplies 'red' part.

```
SELECT DISTINCT supplier.S#  
FROM Supplier,Parts, SP  
WHERE Supplier.S# = SP.S#  
AND SP.P# = Parts.P# AND Parts.color = 'red';
```

vi) Count number of supplier who supplies 'red' part.

```
SELECT count(*)  
FROM (SELECT DISTINCT Supplier.S#  
FROM Supplier,Parts, SP  
WHERE Supplier.S# = SP.S#  
AND SP.P# = Parts.P# AND Parts.color = 'red');
```

vii) Sort the supplier table by sname.

```
SELECT *  
FROM Supplier  
ORDER BY sname;
```

Example 2.22.5 We have following relations :

Supplier (S#, sname, status, city)

Parts (P#, pname, color, weight, city)

SP(S#, P#, quantity)

Answer the following queries in SQL,

i) Delete records in supplier table whose status is 40.

ii) Add one field in supplier table.

Solution :

i) Delete records in supplier table whose status is 40.

```
DELETE FROM Supplier  
WHERE status=40
```

ii) Add one field in supplier table.

```
ALTER TABLE Supplier  
ADD(PhoneNo number);
```

Example 2.22.6 We have following relations :

Supplier (S#, sname, status, city)

Parts (P#, pname, color, weight, city)

SP(S#, P#, quantity)

Answer the following queries in SQL,

i) Find name of parts whose color is 'red'.

ii) Find parts name whose weight less than 10 kg.

iii) Find all parts whose weight from 10 to 20 kg.

- iv) Find average weight of all parts.*
- v) Find S# of supplier who supply part 'p2'.*
- vi) Find name of supplier who supply maximum parts.*
- vii) Sort the parts table by pname.*

Solution :

i) Find name of parts whose color is 'red'.

```
SELECT pname
FROM Parts
WHERE Parts.color = 'red';
```

ii) Find parts name whose weight less than 10 kg.

```
SELECT pname
FROM Parts
WHERE Parts.weight < 10;
```

iii) Find all parts whose weight from 10 to 20 kg.

```
SELECT pname, weight
FROM Parts
WHERE Parts.weight BETWEEN 10 AND 20;
```

iv) Find average weight of all parts.

```
SELECT AVG (weight)
FROM Parts;
```

v) Find S# of supplier who supply part 'p2'.

```
SELECT S#
FROM SP
WHERE p# = 'p2';
```

vi) Find name of supplier who supply maximum parts.

```
SELECT sname, MAX(quantity)
FROM Supplier, Parts, SP
WHERE Supplier.S# = SP.S#
AND SP.P# = P.P#
GROUP BY sname;
```

vii) Sort the parts table by pname.

```
SELECT *
FROM Parts
ORDER BY pname;
```

Example 2.22.7 We have following relations :

Supplier (S#, sname, status, city)

Parts (P#, pname, color, weight, city)

SP(S#, P#, quantity)

Answer the following query in SQL,

Delete records in parts table whose color is 'blue'.

Solution : (i) DELETE FROM Parts

```
WHERE color='blue'
```

Example 2.22.8 Consider following schema and write SQL for given statements.

student (rollno, name, branch)

exam(rollno, subject_code, obtained_marks, paper_code)

papers(paper_code, paper_satter_name, university)

i) Display name of student who got first class in subject '130703'.

ii) Display name of all student with their total mark.

iii) Display list number of student in each university.

iv) Display list of student who has not given any exam.

Solution :

i) Display name of student who got first class in subject '130703'.

```
SELECT name
FROM student,exam
WHERE exam.obtained_marks>60 AND exam.subject_code='130703' AND
student.rollno=exam.rollno
```

ii) Display name of all student with their total mark.

```
SELECT name,SUM(obtained_marks)
FROM student,exam
WHERE student.rollno=exam.rollno
```

iii) Display list number of student in each university.

```
SELECT COUNT(Rollno)
FROM student, exam, papers
WHERE student.rollno=exam.rollno AND exam.paper_code=papers.paper_code;
```

iv) Display list of student who has not given any exam.

```
SELECT name
FROM student NOT IN
(SELECT rollno
FROM student, exam
WHERE student.rollno=exam.rollno)
```

Example 2.22.9 Write down the query for the following table where primary keys are underlined.

Person(ss#, name, address)

Car(license, year, model)

Accident(date, driver, damage-amount)

Owns(ss#, license)

Log(license, date, driver)

1) Find the total number of people whose cars were involved in accidents in 2009.

2) Find the number of accidents in which the cars belonging to "S.Sudarshan"

3) Add a new customer to the database.

4) Add a new accident recorded for the santro belonging to "KORTH"

Solution :

1) Find the total number of people whose cars were involved in accidents in 2009.

```
SELECT count(ss#)
FROM person ,owns,accident,log
WHERE person.ss#=owns.ss# AND
owns.licence=log.licence AND
log.driver=accident.driver AND
accident.date >= '01-jan-2009' and accident.date <='31-dec-09;
```

2) Find the number of accidents in which the cars belonging to "S.Sudarshan"

```
SELECT COUNT(accident.date)
FROM person ,owns,accident,log
WHERE log.date=accident.date AND log.driver=accident.driver AND
own.licence=log.licence AND
person.ss#=own.ss# AND
person.name = 's.sudarshan';
```

3) Add a new customer to the database.

```
INSERT INTO person (ss#, name, address) values (101, 'Madhav', 'Pune');
INSERT INTO owns (ss#, license) values (101, 'L101');
INSERT INTO car (license, year, model) values ('L101', 2017, 'Honda');
INSERT INTO log (license, date, driver) values ('L101', NULL, 'Shankar');
```

4) Add a new accident recorded for the santro belonging to "KORTH"

```
INSERT INTO accident (date, driver, damage-amount) VALUES ('31-Dec-2016', 'Shankar', 10000);
INSERT INTO log (license, date, driver) VALUES ('L111', '31-Dec-2016', 'Shankar');
INSERT INTO car (license, year, model) VALUES ('L111', 2005, 'SANTRO');
INSERT INTO person (ss#, name, address) VALUES (111, 'Korth', 'Mumbai');
INSERT INTO owns (ss#, license) VALUES (111, 'L111');
```

Example 2.22.10 Consider the employee data. Give an expression in SQL for the following query :

Employee(employee_name, street, city)

Works(employee_name, company_name, salary)

Company(company_name, city)

Manages(employee_name, manager_name)

1) Find the name of all employees who work for State Bank.

2) Find the names and cities of residence of all employees who work for State Bank.

3) Find all employee in the database who do not work for State Bank.

4) Find all employee in the database who earn more than every employee of UCO bank.

Solution :

1) Find the name of all employees who work for State Bank.

```
SELECT employee_name
FROM Works
WHERE company_name='State Bank';
```

2) Find the names and cities of residence of all employees who work for State Bank.

```
SELECT employee_name, city
FROM Employee, Works
WHERE company_name='State Bank' AND
Works.employee_name=Employee.employee_name;
```

3) Find all employee in the database who do not work for State Bank.

```
SELECT employee_name
FROM Works
WHERE company_name < > 'State Bank';
```

4) Find all employee in the database who earn more than every employee of UCO bank.

```
SELECT employee_name from Works
WHERE salary > (SELECT MAX(salary) FROM Works
WHERE company_name='UCO bank');
```

Example 2.22.11 Consider following schema and write SQL for given statements.

Student(Rollno, Name, Age, Sex, City).

Student_marks(Rollno, Sub1, Sub2, Sub3, Total, Average)

Write query to

i) Calculate and store total and average marks from sub1, sub2 and sub3.

ii) Display name of students who got more than 60 marks in subject Sub1.

iii) Display name of students with their total and average marks.

iv) Display name of students who got equal marks in subject sub2.

Solution :**i) Calculate and store total and average marks from sub1, sub2 and sub3.**

```
UPDATE Student_marks  
SET Total=sub1+sub2+sub3,  
Average= (sub1+sub2+sub3)/3;
```

ii) Display name of students who got more than 60 marks in subject Sub1.

```
SELECT Name  
FROM Student, Student_marks  
WHERE Student.Rollno=Student_marks.Rollno AND Student_marks.sub1>60
```

iii) Display name of students with their total and average marks.

```
SELECT Name, Total, Average  
FROM Student, Student_marks  
WHERE Student.Rollno=Student_marks.Rollno
```

iv) Display name of students who got equal marks in subject sub2.

```
SELECT Name  
FROM Student S, Student_marks SM1, Student_marks SM2  
WHERE S.Rollno=SM1.Rollno AND SM1.sub2=SM2.sub2;
```

Example 2.22.12 *We have following relations.*

EMP (empno, ename, jobtitle, manager no, hiredate, sal, comm, dept no)

DEPT (dept no, dname, loc)

- i) The employees who are getting salary greater than 3000 for those persons belongings to the department 20*
- ii) Employees who are not getting any commission.*
- iii) Find how many job titles are available in employee table.*
- iv) Display total salary spent for each job category.*
- v) Display number of employees working in each department and their department name.*
- vi) List ename whose manager is NULL.*
- vii) List all employee names and their salaries, whose salary lies between 1500/- and 3500/- both inclusive.*

Solution : i) The employees who are getting salary greater than 3000 for those persons belongings to the department 20

```
SELECT ename  
FROM EMP  
WHERE EMP.sal>3000 AND EMP.dept_no=20
```

ii) Employees who are not getting any commission.

```
SELECT ename
FROM EMP
WHERE EMP.comm IS NULL;
```

iii) Find how many job titles are available in employee table.

```
SELECT count(jobtitle)
FROM EMP
```

iv) Display total salary spent for each job category.

```
SELECT ename,SUM(sal)
FROM EMP
GROUP BY jobtitle;
```

v) Display number of employees working in each department and their department name.

```
SELECT COUNT(EMP.eno), DEPT.dname
FROM EMP, DEPT
WHERE EMP.dept_no=DEPT.dept_no
GROUP BY DEPT.dept_no;
```

vi) List ename whose manager is NULL.

```
SELECT ename
FROM EMP
WHERE manager_no IS NULL;
```

vii) List all employee names and their salaries, whose salary lies between 1500/- and 3500/- both inclusive.

```
SELECT ename, sal
FROM EMP
WHERE sal >= 1500 AND sal <= 3500;
```

Example 2.22.13 *We have following relations.*

EMP (empno, ename, jobtitle, manager no, hiredate, sal, comm, dept no)

DEPT (dept no, dname, loc)

Answer the following queries in SQL

i) Find the employees working in the department 10, 20, 30 only.

ii) Find employees whose names start with letter A or letter a.

iii) Find employees along with their department name.

iv) Find employees whose manager is KING.

v) Find the employees who are working in smith's department.

vi) Find the employees who get salary more than Allen's salary.

vii) Display employees who are getting maximum salary in each department.

Solution : i) Find the employees working in the department 10, 20, 30 only.

```
SELECT empno
FROM EMP
WHERE dept_no BETWEEN 10 AND 30
```

ii) Find employees whose names start with letter A or letter a.

```
SELECT ename
FROM EMP
WHERE ename='A%' OR ename='a%';
```

iii) Find employees along with their department name.

```
SELECT EMP.ename,DEPT.dname
FROM EMP, DEPT
WHERE EMP.dept_no=DEPT.dept_no ;
```

iv) Find employees whose manager is KING.

```
SELECT ename
FROM EMP
WHERE managerno = (SELECT empno FROM EMP WHERE ename='KING');
```

v) Find the employees who are working in smith's department.

```
SELECT ename
FROM EMP,DEPT
WHERE EMP.dept_no=DEPT.dept_no AND ename='Smith';
```

vi) Find the employees who get salary more than Allen's salary.

```
SELECT ename
FROM EMP
WHERE sal> (SELECT sal FROM EMP WHERE ename='Allen');
```

vii) Display employees who are getting maximum salary in each department.

```
SELECT ename, MAX(sal)
FROM EMP
GROUP BY dept_no;
```

Example 2.22.14 Write queries for the following table.

T1 (Empno, Ename, Salary, Designation),

T2 (Empno, Deptno.)

i) Display all rows for salary greater than 5000

ii) Display the deptno for the ename='shyam'.

iii) Add a new column deptname in table T2.

iv) Change the designation of ename='ram' from 'clerk' to 'senior clerk'.

v) Find the total salary of all the rows.

vi) Display Empno, Ename, Deptno and Deptname.

vii) Drop the table T1.

Solution : i) Display all rows for salary greater than 5000

```
SELECT *  
FROM T1  
WHERE Salary > 5000
```

ii) Display the deptno for the ename='shyam'.

```
SELECT deptno  
FROM T1, T2  
WHERE T1.Empno = T2.Empno AND T1.Ename = 'shyam';
```

iii) Add a new column deptname in table T2.

```
ALTER TABLE T2  
ADD (deptname VARCHAR(20));
```

iv) Change the designation of ename='ram' from 'clerk' to 'senior clerk'.

```
UPDATE T1  
SET T1.designation = 'Senior Clerk'  
WHERE T1.ename = 'ram';
```

v) Find the total salary of all the rows.

```
SELECT SUM(Salary)  
FROM T1;
```

vi) Display Empno, Ename, Deptno and Deptname.

```
SELECT E.Empno, E.Ename, D.Deptno, D.Deptname  
FROM T1 E, T2 D  
WHERE E.Empno = D.Empno;
```

vii) Drop the table T1.

```
DROP Table T1;
```

Example 2.22.15 Solve following queries with following table, where underlined attribute is primary key.

primery key.

Person(SS#, name, address)

Car(license, year, model)

Accident(date, driver, damage-amount)

Owns(SS#, license)

Log(licence, date, driver)

i) Find the name of person whose license number is '12345'.

ii) Display name of driver with number of accidents done by that driver.

iii) Add a new accident by 'Ravi' for 'BMW' car on 01/01/2013 for damage amount of 1.5 lakh rupees.

Solution :**i) Find the name of person whose license number is '12345'.**

```
SELECT name
FROM Person, Owns, Car
WHERE Person.SS# = Owns.SS# AND
Car.license = Owns.license AND
Car.license = '12345';
```

ii) Display name of driver with number of accidents done by that driver.

```
SELECT driver, COUNT(*)
FROM Accident
GROUP BY driver
```

iii) Add a new accident by 'Ravi' for 'BMW' car on 01/01/2013 for damage amount of 1.5 lakh rupees.

```
INSERT INTO Person (SS#, name, address) VALUES(111, 'Ravi', 'Mumbai');
INSERT INTO Car (license, year, model) VALUES('L111', '2008', 'BMW');
INSERT INTO Owns (SS#, license) VALUES(111, 'L111');
INSERT INTO Accident ('01/01/2013', 'Ravi', 150000);
INSERT INTO Log ('L111', '01/01/2013', 'Ravi');
```

Example 2.22.16 *For Supplier - Parts database*

Supplier(S#, sname, status, city)

Parts(P#, pname, color, weight, city)

SP(S#, P#, quantity)

Answer the following queries in SQL.

i) Display the name of supplier who lives in 'Ahemdabad'.

ii) Display the parts name which is not supplied yet.

iii) Find all suppliers whose status is either 20 or 30.

Solution :**i) Display the name of supplier who lives in 'Ahemdabad'.**

```
SELECT sname
FROM Supplier
WHERE city = 'Ahemdabad';
```

ii) Display the parts name which is not supplied yet.

```
SELECT pname
FROM Parts, SP
WHERE SP.P# <> Parts.P#
```

iii) Find all suppliers whose status is either 20 or 30.

```
SELECT sname
FROM Supplier
WHERE status = 20 or status = 30;
```

Example 2.22.17 For Supplier - Parts database

Supplier(S#, sname, status, city)

Parts(P#, pname,color,weight,city)

SP(S#, P#, quantity)

Answer the following queries in SQL.

i) Find the name of parts having 'Red' colour.

ii) Delete parts whose weight is more than 100 gram.

iii) Count how many times each supplier has supplied part 'P2'.

iv) How much times shipment is for more than 100 quantities ?

Solution :

i) Find the name of parts having 'Red' colour.

```
SELECT pname
FROM Parts
WHERE color='Red';
```

ii) Delete parts whose weight is more than 100 gram.

```
DELETE FROM Parts
WHERE weight>100
```

iii) Count how many times each supplier has supplied part 'P2'.

```
SELECT S#, COUNT(*)
FROM SP
WHERE P#='P2'
```

iv) How much times shipment is for more than 100 quantities?

```
SELECT S#, COUNT(*)
FROM SP
WHERE quantity>100;
```

Example 2.22.18 Consider following schema and write SQL for given statements.

student(RollNo, Name, Age, Sex, City)

Student_marks(RollNo, Sub1, Sub2, Sub3, Total, Average)

Write query to

i) Display name and city of students whose total marks are greater than 225.

ii) Display name of students who got more than 60 marks in each subject.

iii) Display name of city from where more than 10 students come from.

iv) Display a unique pair of male and female students.

Solution :**i) Display name and city of students whose total marks are greater than 225.**

```
SELECT name, city
FROM student, student_marks
WHERE student.RollNo = student_marks.RollNo AND student_marks.Total > 225
```

ii) Display name of students who got more than 60 marks in each subject.

```
SELECT name
FROM student, student_marks
WHERE student.RollNo = student_marks.RollNo AND
student_marks.Sub1 > 60 OR student_marks.Sub2 > 60 OR student_marks.Sub3 > 60;
```

iii) Display name of city from where more than 10 students come from.

```
SELECT city
FROM student
WHERE count(RollNo) > 10
```

iv) Display a unique pair of male and female students.

```
SELECT S1.name
FROM Student S1, Student S2
WHERE S1.Name = S2.Name AND S1.sex = 'M' S2.sex = 'F'
```

Example 2.22.19 C Write queries for the following tables :

T1 (Empno, Ename, Salary, Designation)

T2 (Empno, Deptno.)

- 1) Display all the details of the employee whose salary is lesser than 10 K.
- 2) Display the Deptno in which employee Seeta is working.
- 3) Add a new column Deptname in table T2.
- 4) Change the designation of Geeta from 'Manager' to 'Senior Manager'.
- 5) Find the total salary of all the employees.
- 6) Display Empno, Ename, Deptno and Deptname
- 7) Drop the table T1.

Solution :**1) Display all the details of the employee whose salary is lesser than 10 K.**

```
SELECT *
FROM T1
WHERE Salary < 10000
```

2) Display the Deptno in which employee Seeta is working.

```
SELECT Deptno
FROM T2, T1
WHERE T1.Empno = T2.Empno AND T1.Ename = 'Seeta';
```

3) Add a new column Deptname in table T2.

```
ALTER TABLE T2  
ADD (Deptname VARCHAR(20));
```

4) Change the designation of Geeta from 'Manager' to 'Senior Manager'.

```
UPDATE T1  
SET Designation = 'Senior Manager'  
WHERE Ename='Geeta';
```

5) Find the total salary of all the employees.

```
SELECT SUM(Salary)  
FROM T1
```

6) Display Empno, Ename, Deptno and Deptname

```
SELECT T1.Empno, T1.Ename, T2.Deptno, T2.Deptname  
FROM T1, T2
```

7) Drop the table T1.

```
DROP TABLE T1
```

Example 2.22.20 We have following relations :

EMP(empno, ename, jobtitle, managerno, hiredate, sal, comm, deptno)

DEPT(deptno, dname, loc)

Answer the following queries in SQL.

- i) Find the Employees working in the department 10, 20, 30 only.*
- ii) Find Employees whose names start with letter A or letter a.*
- iii) Find Employees along with their department name.*
- iv) Insert data in EMP table.*
- v) Find the Employees who are working in Smith's department*
- vi) Update Department name of Department No = 10*
- vii) Display employees who are getting maximum salary in each department*

Solution :

i) Find the Employees working in the department 10, 20, 30 only.

```
SELECT ename  
FROM EMP  
WHERE deptno BETWEEN 10 AND 30
```

ii) Find Employees whose names start with letter A or letter a.

```
SELECT ename  
FROM EMP  
WHERE ename='A%' OR ename='a%';
```


iii) Find Employees along with their department name.

```
SELECT EMP.ename,DEPT.dname
FROM EMP, DEPT
WHERE EMP.deptno=DEPT.deptno ;
```

iv) Insert data in EMP table.

```
INSERT INTO EMP( empno, ename, jobtitle, managerno, hiredate, sal, comm, deptno)
VALUES ('E111','AAA','Manager',M123,01-01-2010,20000,2000,'D111');
```

v) Find the Employees who are working in Smith's department

```
SELECT ename
FROM EMP,DEPT
WHERE EMP.deptno=DEPT.deptno AND ename='Smith';
```

vi) Update Department name of Department No = 10

```
UPDATE DEPT
SET dname='Accounts'
WHERE deptno=10;
```

vii) Display employees who are getting maximum salary in each department

```
SELECT ename, MAX(sal)
FROM EMP
GROUP BY deptno;
```

Example 2.22.21 Consider following Hotel database, primary keys are underlined :

hotel(hotel_no,name,type,price)

room(room-no,hotel-no,type,price)

booking(hotel-no,guest-no,date-from,date-to,room-no)

guest(guest-no,name,address)

Give an expression in SQL for each of the following queries

- (1) List the names and addresses of all guests in London, alphabetically ordered by name.
- (2) List out hotel name and total number of rooms available
- (3) List the details of all the rooms at the Grosvenor Hotel, including the name of the guest staying in the room, if the room is occupied.
- (4) List all guests currently staying at the Grosvenor Hotel.
- (5) List the rooms that are currently unoccupied at the Grosvenor Hotel.
- (6) List the number of rooms in each hotel in London.
- (7) List out all guests who have booked room for three or more days.

Solution :**(1) List the names and addresses of all guests in London, alphabetically ordered by name.**

```
SELECT name, address
FROM guest
WHERE address LIKE '%London%'
ORDER BY name;
```

(2) List out hotel name and total number of rooms available

```
SELECT name, COUNT(room-no)
FROM hotel, room
WHERE hotel.hotel_no=room.hotel_no
GROUP by hotel_no;
```

(3) List the details of all the rooms at the Grosvenor Hotel, including the name of the guest staying in the room, if the room is occupied.

```
SELECT r.* FROM Room r LEFT JOIN
(SELECT g.guestName, h.hotelNo, b.roomNo
FROM guest g, booking b, hotel h
WHERE g.guest_no = b.guest_no AND
b.hotel_no = h.hotel_no AND
h.name= 'Grosvenor Hotel' AND
date_from <= CURRENT_DATE AND
date_to >= CURRENT_DATE)
AS XXX ON r.hotel_no = XXX.hotel_no AND r.room_no = XXX.room_no;
```

(4) List all guests currently staying at the Grosvenor Hotel.

```
SELECT * FROM guest
WHERE guest_no =
(SELECT guest_no FROM booking
WHERE
date-from <= CURRENT_DATE AND date-to >= CURRENT_DATE AND
hotel_no =
(SELECT hotel_no FROM hotel
WHERE name = 'Grosvenor Hotel'));
```

(5) List the rooms that are currently unoccupied at the Grosvenor Hotel.

```
SELECT (r.hotel_no, r.room_no, r.type, r.price)
FROM room r, hotel h
WHERE r.hotel_no = h.hotel_no AND
h.name = 'Grosvenor Hotel' AND
NOT EXIST
(SELECT *
FROM booking b, hotel h
WHERE (date_from <= 'CURRENT_DATE'
AND date_to >= 'CURRENT_DATE')
AND r.hotel_no=b.hotel_no
AND r.room_no=b.room_no
```

```
AND r.hotel_no=h.hotel_no
AND name = 'Grosvenor Hotel');
```

(6) List the number of rooms in each hotel in London.

```
SELECT hotel_no, COUNT(room_no) AS count
FROM room r, hotel h
WHERE r.hotel_no = h.hotel_no AND city = 'London'
GROUP BY hotel_no;
```

(7) List out all guests who have booked room for three or more days.

```
SELECT guest-no, name
FROM guest g ,booking b
WHERE b.date-to Minus b.date-from >=3
```

Example 2.22.22 Consider following schema and write SQL for given statements

employee(employee-name,street,city)

works(employee-name, company-name, salary)

company(company-name, city)

manages(employee-name,manager-name)

(1) Find the names of all employees who work for first bank corporation.

(2) Give all employees of first bank corporation a 10-percent raise.

(3) Find the names and cities of residence of all employees who work for first bank corporation.

(4) Find the names and street addresses, cities of residence of all employees who work for First Bank Corporation and earn more than \$10,000

(5) Find all employees in the database who live in the same cities as the companies for which they work.

(6) Find all employees in the database who do not work for First Bank Corporation.

(7) Find the company and number of employees in company that has more than 30 employees

Solution :

(1) Find the names of all employees who work for First Bank Corporation.

```
SELECT employee_name
FROM works
WHERE company_name='First Bank Corporation';
```

(2) Give all employees of First Bank Corporation a 10-percent raise.

```
UPDATE works
SET salary=salary*1.1
WHERE company_name='First Bank Corporation';
```

(3) Find the names and cities of residence of all employees who work for First Bank Corporation.

```
SELECT employee_name,city
FROM employee
WHERE employee_name IN
(SELECT employee_name
FROM works
WHERE company_name='First Bank Corporation');
```

(4) Find the names and Street addresses, cities of residence of all employees who work for First Bank Corporation and earn more than \$10,000

```
SELECT employee_name,city
FROM employee
WHERE employee_name IN
(SELECT employee_name
FROM works
WHERE company_name='First Bank Corporation' AND salary>10000);
OR
SELECT E.employee_name, E.street, E.city
FROM employee as E, works as W
WHERE
E.employee_name=W.employee_name AND
W.company_name='First Bank Corporation' AND W.salary>10000
```

(5) Find all employees in the database who live in the same cities as the companies for which they work.

```
SELECT E.employee_name
FROM employee as E, works as W, company as C
where E.employee_name=W.employee_name AND
E.city=C.city AND
W.company_name=C.company_name;
```

(6) Find all employees in the database who do not work for First Bank Corporation.

```
SELECT employee_name
FROM works
WHERE company_name <>'First Bank Corporation';
```

(7) Find the company and number of employees in company that has more than 30 employees

```
SELECT company_name,COUNT(employee_name)
FROM employee E, company C,works W
WHERE E.employee_name=W.employee_name AND
W.company_name=C.company_name
HAVING COUNT(employee_name)>30;
```

Example 2.22.23 Consider following relations and write SQL queries for given statements

Assume suitable constraints

Instructor(ID, Name, Dept_name, Salary)

Teaches(ID, Course_id, Sec_id, Semester(even/odd), Year)

(1) Write SQL query to create Instructor table

(2) Find the average salary of the instructor in computer department.

(3) Find the number of instructors in each department who teach a course in even semester of 2016

(4) Find the names of instructor with salary amounts between 30000 and 50000

Solution : (1) Write SQL query to create Instructor table

```
CREATE TABLE Instructor
(ID CHAR,
Name VARCHAR(20),
Dept_name VARCHAR(15),
Salary numeric(8,2)
);
```

(2) Find the average salary of the instructor in computer department.

```
SELECT AVG(Salary)
FROM Instructor
WHERE Dept_name='Computer';
```

(3) Find the number of instructors in each department who teach a course in even semester of 2016

```
SELECT COUNT(DISTINCT ID)
FROM Teaches
WHERE Semester='even' AND Year=2016
```

(4) Find the names of instructor with salary amounts between 30000 and 50000

```
SELECT Name
FROM Instructor
WHERE Salary >=30000 AND Salary <=50000
```

Example 2.22.24 Consider following schema and write SQL for given statements

Client_master(clientno,name,address,city,pincode,state,baldue)

Product_master(productno,name,profitpercent,unitmeasure,sellprice,costprice)

Sales_master(Salesmanno,name,address,city,pincode,state,salary,tgtotget,remarks)

(1) Find out the names of all clients

(2) List all the clients who are located in Mumbai

(3) Delete all salesmen from salesman_master whose salaries are equal to ₹3500

(4) Destroy the table client_master along with data

- (5) List the names of all clients having 'a' as the second letter in their names.
- (6) Count the number of products having cost price is less than or equal to 500.
- (7) Calculate the average, minimum and maximum Sell price of product

Solution :

```
(1) SELECT name FROM Client_master;
(2) SELECT * FROM Client_master
    WHERE city='Mumbai'
(3) DELETE FROM Salesman_master
    WHERE salary=3500;

(4) DROP TABLE Client_master;
(5) SELECT name
FROM Client_master
WHERE name like ' _a%';
(6) SELECT count(productno)
FROM product_master
WHERE costprice <= 500
(7) SELECT AVG(sellprice), MIN(sellprice), MAX(sellprice)
FROM product_master;
```

Example 2.22.25 Assume the following table.

Degree (degcode, name, subject)

Candidate (seatno, degcode, name, semester, month, year, result)

Marks (seatno, degcode, semester, month, year, papcode, marks)

[degcode – degree code, name – name of the degree (Eg. MSc.), subject – subject of the course (Eg. Physis), papcode – paper code (Eg. A1)]

Solve the following queries using SQL;

Write a SELECT statement to display,

- (i) all the degree codes which are there in the candidate table but not present in degree table in the order of degcode.
- (ii) the name of all the candidates who have got less than 40 marks in exactly 2 subjects.
- (iii) the name, subject and number of candidates for all degrees in which there are less than 5 candidates.
- (iv) the names of all the candidate who have got highest total marks in MSc. Maths.

Solution : (i) SELECT C.degcode

```
FROM Candidate C,
WHERE NOT EXISTS
    (SELECT D.degcode
    FROM Degree D
    WHERE D.degcode=C.degcode)
ORDER by C.degcode
```

(ii) SELECT C.name

```
FROM Candidate C, Degree D, Marks M
WHERE
C.seatno=M.seatno AND C.degcode=D.degcode AND C.degcode=M.degcode AND M.marks < 40
GROUP BY C.seatno
HAVING count(D.subject)=2;
```

(iii) SELECT D.name,D.subject,count(*)

```
FROM degree D, Candidate C
WHERE D.degcode=C.degcode
HAVING( SELECT count(*) FROM Candidate <5);
```

(iv) SELECT C.name

```
FROM Candidate C, Degree D, Marks M
WHERE
D.degname='MSc' AND D.subject='Maths' AND C.degcode=D.degcode AND C.seatno=M.seatno
AND
M.marks = (SELECT max(M.marks) FROM Marks M)
```

Example 2.22.26 Consider a student registration database comprising of the below given table schema.

Student File

Student Number	Student Name	Address	Telephone
----------------	--------------	---------	-----------

Course File

Course Number	Description	Hours	Professor Number
---------------	-------------	-------	------------------

Professor File

Professor Number	Name	Office
------------------	------	--------

Registration File

Student Number	Course Number	Date
----------------	---------------	------

Consider a suitable sample of tuples / records for the above mentioned tables and write DML statements (SQL) to answer for the queries listed below.

- i) Which courses does a specific professor teach ?
- ii) What courses are taught by two specific professors ?
- iii) Who teaches a specific course and where is his/her office ?
- iv) For a specific student number, in which courses is the student registered and what is his/her name ?
- v) Who are the professors for a specific student ?
- vi) Who are the students registered in a specific course ?

Solution :**(i)**

```
SELECT P.name,C.description
FROM Professor P, Course C
WHERE P.ProfessorNumber=C.ProfessorNumber
HAVING count(DISTINCT P.name)=2
```

(ii)

```
SELECT P.name,C.description
FROM Professor P, Course C
WHERE P.ProfessorNumber=C.ProfessorNumber
```

(iii)

```
SELECT P.name,P.office, C.description
FROM Professor P, Course C
WHERE P.ProfessorNumber=C.ProfessorNumber
```

(iv)

```
SELECT S.StudentNumber,S.StudentNumber,C.Description
FROM Student S, Course C, Registration R
WHERE S.StudentNumber=R.StudentNumber AND C.CourseNumber=R.CourseNumber
```

(v)

```
SELECT S.StudentName, P.Name
FROM Student S, Course C, Professor P, Registration R
WHERE C.ProfessorNumber=P.ProfessorNumber
      AND C.CourseNumber=R.CourseNumber
      AND S.StudentNumber=R.StudentNumber
GROUP BY P.ProfessorNumber
```

(vi)

```
SELECT S.StudentName, C.Description
FROM Student S, Course C, Registration R
WHERE S.StudentNumber=R.StudentNumber
      AND R.CourseNumber=C.CourseNumber
GROUP BY C.CourseNumber
```

Example 2.22.27 Consider following RESORT database*RESORT(resortno, resortname, resorttype, resortaddr,resortcity,numsuite)**SUITE (suiteño, resortno, suiteprice)**RESERVATION (reservationno, resorttype, resortaddr, resortcity , numsuite)**VISITOR(visitno, firstname, lastname, visitoraddr)*

- i) Write the SQL to list full details of all the resorts on Los Angeles.
- ii) Write the SQL to list full details of all resorts having number of suites more than 30.
- iii) Write the SQL to list visitors in ascending order by firstname.

Solution : (i) SELECT * FROM RESORT WHERE resortcity ='Los Angeles';
(ii) SELECT * FROM RESORT WHERE numsuite>30;
(iii) SELECT * FROM VISITOR ORDER BY firstname;

Example 2.22.28 Consider the following database and answer the queries-

WORKS(Pname,Cname,Salary)

LIVES(Pname, Street, City)

LOCATED_IN(Cname, City)

Write the following queries in SQL :

- (i) List the names of the people who work for the company 'Wipro' along with cities they live in.*
- (ii) Find the names of the persons who do not work for 'Infosys'*
- (iii) Find the people whose salaries are more than that of all of the 'oracle' employees.*
- (iv) Find the persons who works and lives in the same city.*

Solution :

(i) SELECT L.Pname, L.City

FROM Works W, Lives L

Where W.Cname = "Wipro" and W.Pname = L.Pname;

(ii) SELECT Pname

FROM WORKS

WHERE Cname != 'Infoysy';

(iii) SELECT Pname

FROM WORKS

WHERE Salary > ALL

(SELECT Salary

FROM WORKS

WHERE Cname = "Oracle");

(iv) SELECT W.Pname

FROM WORKS W, LIVES L, LOCATED_IN I

WHERE W.Cname = I.Cname AND W.Pname = L.Pname

AND I.City = L.City

Example 2.22.29 Write the SQL queries for the following relational schema :

Sailors(Sid, Sname, Rating, Age)

Boats(Bid, Bname,color)

Reserve(Sid, Bid, Day)

- i) Retrieve the Sailor's name who have reserved red and green boat
- ii) Retrieve the no of boats which are not reserved.
- iii) Retrieve the Sailors name who have reserved boat number 103
- iv) Retrieve the Sailors name who have reserved all boats.

Solution : (i)

```
SELECT S.sname
FROM Sailors S, Reserves R, Boats B
WHERE S.sid=R.sid
AND R.bid=B.bid
AND ( B.color='red' AND B.color='green')
```

(ii)

```
SELECT Sname
FROM Sailors
WHERE Sid NOT IN
(SELECT Sid
FROM Reserve, Sailors
WHERE Reserve.sid=Sailors.sid )
```

(iii)

```
SELECT S.sname
FROM Sailors S, Reserves R
WHERE S.sid=R.sid AND bid=103
(iv) SELECT S.sname
FROM Sailors S
WHERE NOT EXISTS (( SELECT B.bid
FROM Boats B )
EXCEPT
( SELECT R.bid
FROM Reserves R
WHERE R.sid=S.sid ))
```

Example 2.22.30 Write SQL queries for the following relational schema

CUSTOMER(CID, CNAME,EMAIL,ADDR,PHONE)

ITEM(ITEM_NO,ITEM_NAME,PRICE, BRAND)

SALES(CID,ITEM_NO,#ITEMS, AMOUNT,SALE_DATE)

SUPPLIER(SID, SNAME, SPHONE, SADDR)

SUPPLY(SID, ITEM_NO, SUPPLY_DATE, QTY)

- (i) List the items purchased by Customer 'Prasanth'
- (ii) Retrieve items supplied by all suppliers starting from 1st Jan 2019 to 30th Jan 2019.

- (iii) Get the details of customers whose total purchase of items worth more than 5000 rupees.
- (iv) List total sales amount, total items, average sale amount of all items.
- (v) Display customers who have not purchased any items

Solution : (i)

```
SELECT ITEM_NAME
FROM ITEM I,CUSTOMER C and SALES S
WHERE C.CID= S.CID AND I.ITEM_NO = S.ITEM_NO AND C.CNAME='Prasanth'
```

(ii)

```
SELECT ITEM_NAME
FROM ITEM, SUPPLIER, SUPPLY
WHERE ITEM.ITEM_NO=SUPPLY.ITEM_NO AND
AND SUPPLY.SID=SUPPLIER.SID AND
SUPPLY.SUPPLY_DATE '1 JAN 2019' AND '30 JAN 2019'
```

(iii)

```
SELECT CID, CNAME, EMAIL,ADDR,PHONE
FROM CUSTOMER,SALES
WHERE CUSTOMER.CID=SALES.CID AND SALES.AMOUNT>=5000
```

(iv)

```
SELECT
    SUM(AMOUNT),#item,AVG(AMOUNT)
FROM SALES
WHERE SALES.ITEM_NO = ITEM.ITEM_NO
```

(v)

```
SELECT CID, CNAME, EMAIL,ADDR,PHONE
FROM CUSTOMER
WHERE CID
NOT IN (
SELECT CID
FROM SALES
)
```

Example 2.22.31 Consider following database

Student(Roll_No, Name, Address)

Subject(Sub_code, Sub_name)

Marks(Roll_no, Sub_code, Marks)

Write following queries in SQL

- 1) Find average marks of each student along with the name of student
 - 2) Find how many students have failed in the subject "DBMS"
- (note:failed means obtained less than 40 marks)

Solution : (1)

```
SELECT Name, AVG(Marks)
FROM Student,Marks
WHERE Student.Roll_No=Marks.Roll_No
```

(2)

```
SELECT COUNT(Roll_NO)
FROM Subject,Marks
WHERE Marks.Marks<40 AND Subject.sub_code=Marks.Sub_code AND Sub_name="DBMS"
```

Example 2.22.32 Consider the following six relations for an order processing database application in a company.

customer (cust #, cname, city)

order (order #, odate, cust #, ord-Amt)

order-item (order #, item #, Qty)

item (item #, unit-price)

shipment (order #, warehouse #, ship-date)

Warehouse (warehouse #, city)

Here, old-Amt refers to total amount of an order; odate is the date the order was placed; ship-date is the date an order is shipped from the warehouse. Assume that an order can be shipped from several warehouses.

Specify the following in SQL queries

- i) List the order # and ship-date for all orders shipped from warehouse number 'W2'.
- ii) List the warehouse information from which the customer named 'Jose Lopez' was supplied his orders.
- iii) List the orders that were not shipped within 30 days of ordering.
- iv) List the order # for orders that were shipped from all warehouses that the company has in New York.

Solution:**i)**

```
SELECT Order#, ship-date
FROM shipment
WHERE warehouse# = 'W2'
```

ii)

```
SELECT order.order#, shipment.warehouse#
FROM customer, shipment, order
WHERE order.order# = shipment.order#
      AND customer.cust# = order.cust#
      AND customer.cname = 'Jose Lopez'
```

iii)

```
SELECT order#, odate, cust#, ord-Amt
FROM order, shipment,
WHERE order.order#=shipment.order# AND shipment.ship-date>30
```

iv)

```
SELECT shipment.order#
FROM shipment, Warehouse
WHERE shipment.warehouse# = Warehouse.warehouse#
AND Warehouse.city = "New York"
```

Example 2.22.33 Consider the following database schema :*Physician (reg_no, name, tel_no, city)**Patient(p_name, street, city)**Visit (p_name, reg_no, date_of_visit, fee)*

Write SQL queries for following requirements (any two)

i) Find the name and city of patients who visited a physician on 13 July 2017.

ii) Get the name of the physician and the total no. of patients visited him.

iii) Get the details of date wise fees collected at clinic.

SPPU : Aug 17, End Sem, Marks 5**Solution :**

- ```
(i) SELECT p_name, city
FROM Patient, Visit
WHERE Patient.p_name=Visit.p_name AND date_of_visit='13-July-2017'
(ii) SELECT name, count(*)
FROM Physician, Visit
WHERE Physician.reg_no = Visit.reg_no
GROUP BY Physician.reg_no
(iii) SELECT *
FROM Visit
GROUP BY date_of_visit
```

**Example 2.22.34** Consider insurance database with following schema :*person (driver-id, name, address)**car (license, model, year)**accident (report-no, date, location)**owns (driver-id, license)**participated (driver-id, car, report-no, damage-amount)*

Write a query in SQL for following requirements (any two) :

- i) Find the total no. of people who owned cars that were involved in accidents in 2016.
- ii) Retrieve the name of person whose address contains Pune.
- iii) Find the name of persons having more than two cars

**SPPU : Dec 17, End Sem, Marks 5**

**Solution :**

- i) 

```
SELECT count(DISTINCT name)
FROM person, accident, participated
WHERE accident.report-no = participated.report-no AND participated.driver-id =
person.driver-id AND date BETWEEN DATE '2016-00-00' AND DATE '2016-12-31'
```
- ii) 

```
SELECT name
FROM person
WHERE address LIKE %Pune'
```
- iii) 

```
SELECT name
FROM person
WHERE 2 <
(SELECT count(*)
FROM person,car,owns
WHERE car.license = owns.license AND owns.driver-id=person.driver-id
```

**Example 2.22.35** Schema definition for supplier-and-parts database.

Supplier = (supplier\_number, supplier\_name, status, city)

Parts = (part\_number, part\_name, colour, weight, city)

Shipments = (supplier\_number, part\_number, quantity)

Write SQL query for following requirements (any 2).

- i) Find shipment information (supplier\_number, supplier\_name, part\_number, part\_name, quantity) for those having quantity less than 150.
- ii) List supplier\_number, supplier\_name, part\_number, part\_name for those suppliers who made shipment of parts whose quantity is larger than the average quantity.
- iii) Find aggregate quantity of part\_number 'A692' of colour 'GREEN' for which shipment made by supplier\_number who reside in 'MUMBAI'

**SPPU : May 18, End Sem, Marks 5**

**Solution :**

- i) 

```
SELECT supplier_number,supplier_name,part_number,part_name,quantity
FROM Supplier, Parts, Shipments
WHERE Supplier.supplier_number = Shipments.supplier_number AND Parts.part_number
=Shipments.part_number
AND Shipments.quantity<150
```
- ii) 

```
SELECT supplier_number, supplier_name, part_number, part_name
FROM Supplier, Parts
WHERE Supplier.supplier_number = Shipments.supplier_number AND Part.part_number=
```

```
Shipments.part_number AND
Shipments.quantity>(SELECT AVG(quantity) FROM Shipments)
iii) SELECT COUNT(part_number)
FROM Parts, Supplier, Shipments
WHERE Supplier.supplier_number = Shipments.supplier_number AND Part.part_number=
Shipments.part_number AND
Parts.part_number = 'A692' AND Parts.colour='GREEN' AND Supplier.city='MUMBAI'
```

**Example 2.22.36** Consider the following database schema :

*Loan (loan\_no, Branch\_name, Amount)*

*Borrower (Cust\_name, Loan\_no).*

Write SQL queries for following requirements (any two) :

- i) Find all customers who have a loan from bank. Find their names, loan nos. and Loan amount.
- ii) Find names of customers in alphabetical order who have a loan at Pune branch.
- iii) Find all loan nos. for loan made at Pune branch with loan amount greater than 20000.

**SPPU : Oct 18, End Sem, Marks 5**

**Solution :**

- (i) SELECT Cust\_name Loan\_no, Amount  
FROM Loan, Borrower  
WHERE Loan.Loan\_no = Borrower.Loan\_no
- (ii) SELECT Cust\_name  
FROM Borrower, Loan  
WHERE Loan.Branch\_name = 'Pune'  
ORDER BY Cust\_name ASC
- (iii) SELECT Loan\_no  
FROM Loan, Borrower  
WHERE Loan.Loan\_no = Borrower.Loan\_no AND Loan.Branch\_name = 'Pune' AND  
Loan.Amount>20000

**Example 2.22.37** Consider Employee database with following schema :

*Employee(Emp\_Id, First\_Name, Last\_Name, Salary, Joining\_Date, Department)*

*Bonus(Emp\_Ref\_Id, Bonus\_Amount, Bonus\_Date)*

*Designation(Emp\_Ref\_Id, Emp\_Designation, Affected\_From)*

Write queries in SQL for following requirements (any 2):

- i) To fetch the departments that have less than five people in it.
- ii) To print the name of employees having the highest salary in each department.
- iii) Write an SQL query to print details of the employee who are also Managers.

**SPPU : Dec 18, End Sem, Marks 5**

**Solution :**

- (i) 

```
SELECT Department, COUNT(Emp_Id) as 'Number of People'
FROM Employee GROUP BY Department
HAVING COUNT(Emp_Id) <5
```
- (ii) 

```
SELECT E.Department,t.First_Name,t.Salary
FROM
 (SELECT MAX(Salary) AS TotalSalary, Department FROM Employee GROUP BY Department)
 AS temp
 INNER JOIN ON temp.Department=E.Department AND temp.TotalSalary=E.Salary;
```
- (iii) 

```
SELECT DISTINCT E.FIRST_NAME, D.Emp_Designation
FROM Employee E, Designation D
WHERE E.Emp_Id = D.Emp_Ref_Id
AND D.Emp_Designation IN ('Manager');
```

**Example 2.22.38** Consider following schema :

*account (acct-no, branch - name, balance)*

*Depositor (cust - name, acct - no)*

*borrower (cust-name, loan-no)*

*loan (loan - no, branch - name, amount)*

*Write following queries using SQL (any 2)*

*i) Find names of all customers who have a loan at the redwood branch.*

*ii) Find all customers who are having an account and loan or both.*

*iii) Find average account balance at each branch.*

**SPPU : May 19, End Sem, Marks 5**

**Solution :**

- (i) 

```
SELECT DISTINCT cust-name
FROM borrower, loan
WHERE borrower.loan-no = loan.loan-no AND loan.branch-name = 'redwood'
```
- (ii) 

```
SELECT cust-name FROM Depositor
UNION
SELECT cust-name FROM borrower
```
- (iii) 

```
SELECT branch-name, AVG(balance)
FROM account
GROUP BY branch-name
```

**Example 2.22.39** Consider the following database schema :

*Emp(E\_number, E\_name, Dept\_no)*

*Dept(Dept\_no, Dept\_name)*

*Address(Dept\_name, Dept\_location)*



Write SQL queries for following requirements (any 2)

- i) Display the name of department for the employee having E\_number 'E1011'.
- ii) Display the location of department where employee 'Ramesh' is working.
- iii) Display total no. of employees working in each department.

**SPPU : Oct 19, In Sem, Marks 5**

**Solution :**

- i) 

```
SELECT Dept-name
FROM Dept, Emp
WHERE Dept.Dept_no = Emp.Dept_no AND E_number = 'E1011'
```
- ii) 

```
SELECT Dept_location
FROM Address A, Dept D, Emp E
WHERE E.E_name = 'Ramesh' AND E.Dept_no = D.Dept_no AND
Dept.Dept_name = A.Dept_name
```
- iii) 

```
SELECT COUNT(Emp_no)
FROM Emp E, Dept D
WHERE E.Dept_no = D.Dept_no
```

**Example 2.22.40** Emp(E\_number, E\_name, Dept\_no)

Dept(Dept\_no, Dept\_name).

Consider the schema given above. Consider above tables are created without considering the Dept\_no as primary key in Dept table and foreign key in Emp table. Assuming tables are already created write SQL queries for following requirements.

- i) Create primary key in dept table considering above situations.
- ii) Create foreign key considering EMP as child table and dept as master table also consider the above situation.
- iii) Add column salary with appropriate data type in EMP table.

**SPPU : Oct 19, In Sem, Marks 5**

**Solution :**

- i) 

```
ALTER TABLE Dept
ADD PRIMARY KEY(Dept_no)
```
- ii) 

```
ALTER TABLE Emp
ADD CONSTRAINT FK_deptno
FOREIGN KEY (Dept_no) REFERENCES Dept(Dept_no);
```
- iii) 

```
ALTER TABLE Emp
ADD Salary int
```



## Notes

This image shows a full page of blank white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page, typical of notebook or legal stationery. There are no margins, text, or other markings on the page.